

Received 12 November 2025, accepted 25 November 2025, date of publication 3 December 2025,
date of current version 11 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3638863

RESEARCH ARTICLE

Efficient Routing for Software-Defined Wireless Sensor Networks: A Naïve Bayes Approach

AMINE TCHERAK¹, SAMIA LOUCIF², (Senior Member, IEEE),
AND MOHAMED OULD KHAOUA¹

¹LRDSI, Department of Computer Science, University of Blida 1, Blida 09000, Algeria

²College of Technological Innovation, Zayed University, Abu Dhabi, United Arab Emirates

Corresponding author: Amine Tcherak (tcherak_amine@univ-blida.dz)

ABSTRACT Wireless Sensor Networks (WSNs) form the backbone of Internet of Things (IoT) applications. Software-Defined Networking (SDN) is an emerging networking paradigm that extends the lifetime of WSNs by transferring the resource-intensive routing task from sensor nodes to a centralized controller. However, many SDN-based routing schemes for WSNs employ inefficient algorithms at the controller. Traditional shortest-path methods often create traffic imbalances across neighboring nodes, while Reinforcement Learning (RL)-based approaches typically generate excessive control traffic. Both issues accelerate energy depletion and reduce network lifetime. Moreover, existing algorithms frequently overlook critical factors, such as buffer occupancy, when selecting relay nodes, which can lead to congestion, packet loss, and increased latency. To address these limitations, this paper proposes NBSDN, a Naïve Bayes-based routing algorithm for SDN-enabled WSNs. NBSDN extends our earlier energy- and distance-aware solution (referred to as NB-SDWSN) by incorporating link quality and node buffer capacity into the routing decision process. The controller periodically rotates relay node assignments among neighboring nodes based on their residual energy, distance to the sink, link quality, and buffer occupancy, thereby balancing load and preventing congestion. Extensive simulations conducted in the COOJA environment demonstrate that NBSDN significantly outperforms benchmark algorithms—including the Dijkstra-based Shortest Path SDN (SPSDN), Energy-Aware SDN (EASDN), NB-SDWSN, and RL-based SDN (RLSDN) routing—in terms of network lifetime, control overhead, packet loss, latency, and throughput.

INDEX TERMS WSNs, IoT, SDN, Naïve Bayes algorithm, load balancing, congestion avoidance, COOJA simulator, performance analysis.

I. INTRODUCTION

The Internet of Things (IoT) refers to a network of physical items embedded with sensors, actuators, electronics, software, and network capabilities, enabling them to collect and exchange data with each other and with other devices and systems via the Internet. These items span from vehicles and household appliances to wearable devices and industrial machinery. By connecting these objects to the Internet, they acquire enhanced functionalities, becoming increasingly intelligent and responsive, thereby offering a spectrum of applications across various domains, including

smart homes, smart agriculture, healthcare monitoring, environmental surveillance, and natural disaster detection [1].

At the core of IoT lies the Wireless Sensor Network (WSN), which is composed of small, wirelessly connected devices called sensor nodes, as depicted in Fig. 1. These nodes, equipped with embedded sensors, are distributed throughout a designated area and are tasked with periodically capturing diverse environmental and physical parameters such as temperature, humidity, heartbeat, and blood pressure. Depending on the IoT application, the collected data originates either from the surrounding environment where the sensor nodes are deployed or from the entities to which the nodes are attached. Subsequently, the gathered data is transmitted, either directly or via intermediary nodes, to the

The associate editor coordinating the review of this manuscript and approving it for publication was Chan Hwang See¹.

TABLE 1. List of acronyms.

Acronym	Definition
ABC	Artificial Bee Colony
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
DRL	Deep Reinforcement Learning
EASDN	Energy-Aware Software-Defined Networking
GA	Genetic Algorithm
GWO	Grey Wolf Optimizer
ICA	Imperialist Competitive Algorithm
IoT	Internet of Things
ML	Machine Learning
NB-SDWSN	Naïve Bayes for Software-Defined Wireless Sensor Networks
NBSDN	Naïve Bayes-based routing for Software-Defined Networking
PSO	Particle Swarm Optimization
RL	Reinforcement Learning
RLSDN	Reinforcement Learning-based routing for Software-Defined Networking
RSSI	Received Signal Strength Indicator
SDN	Software-Defined Networking
SDN-WISE	Software Defined Networking in Wireless Sensor Networks
SDWSN	Software-Defined Wireless Sensor Network
SPSDN	Shortest Path-based routing for Software-Defined Networking
UDGM	Unit Disk Graph Medium
WOA	Whale Optimization Algorithm
WSN	Wireless Sensor Network

sink node. This pivotal node, functioning as a gateway, transmits the received data over established infrastructures like the Internet to a remote server where it is stored. Through dedicated applications, users can remotely access and visualize the recorded data, offering real-time remote monitoring of the conditions and status of the subjects or areas where the nodes are placed.

Sensor nodes, which are often characterized by their constrained resources, operate within energy limitations due to their reliance on non-replaceable batteries. This predicament primarily stems from the deployment of these nodes in harsh and inaccessible environments, thereby restricting their operational capacities. Moreover, their communication range is confined to a limited number of neighboring nodes due to the restricted coverage of their wireless communication modules. Additionally, the processing power and memory of these nodes are constrained, resulting in reduced computational and storage capabilities. Functionally, sensor nodes perform three primary tasks: first, they continuously acquire sensory data,

either through their sensors or via data shared by adjacent nodes. Second, they periodically execute a routing protocol to identify and select the best neighbor to act as a relay node for transmitting data to the sink. This process involves identifying the nearest neighboring node to the sink, i.e., the neighboring node that requires passing through the minimum number of hops or intermediate sensor nodes to reach the sink. Third, the sensor nodes are responsible for relaying the collected data to the sink via the identified relay node.

The routing process frequently demands significant utilization of the sensor node's computational power and memory, thus hastening the depletion of its battery. Energy exhaustion leads to node shutdown, thereby shortening the network lifespan. Additionally, the failure of a sensor node can result in the isolation of its neighboring nodes, particularly if it serves as their exclusive pathway to the sink. Consequently, they are unable to transmit their data to the sink, escalating packet loss and diminishing the overall network throughput. To mitigate this challenge, researchers have introduced a new networking paradigm to WSNs called Software-Defined Networking (SDN) [2], resulting in a centralized network, widely referred to as the Software-Defined Wireless Sensor Network (SDWSN). The SDWSN is made up of two logically separated layers named planes: the control plane and the data plane. The control plane represents the central point of the SDWSN and entirely manages and controls the WSN, which is the data plane. The control plane consists of a server known as the controller. The controller continuously maintains an entire view of the WSN topology through control packets that are sent by the nodes within the WSN. This enables the controller to handle the routing process instead of the sensor nodes by periodically calculating and assigning to each node its best relay node (best neighbor) responsible for forwarding its packets toward the sink. The sensor nodes then learn their relay node via control packets sent from the controller. Consequently, the heavy routing process is removed from the nodes, preserving their resources and prolonging the network lifetime.

Several SDWSN studies have proposed controller-side routing algorithms. In [3], [4], and [5], the authors used shortest-path algorithms such as Dijkstra or Bellman-Ford. These methods assume that a sensor's best relay is the neighbor closest to the sink. However, when a node has many neighbors, this choice forces all traffic through a single relay. As a result, that relay must receive, store, process, and forward all packets, rapidly depleting its energy and shortening network lifetime. Concentrating traffic on one neighbor can also cause congestion: once its buffer becomes full, additional packets are dropped. This increases packet loss and ultimately reduces overall network throughput.

Other studies have introduced controller-based routing algorithms, including the Energy-Aware SDN (EASDN) method [6] and several Reinforcement Learning (RL)-driven schemes [7], [8], [9], [10] based on the principles in [11]. Unlike shortest-path routing, these approaches rotate the relay role among a node's neighbors according to their

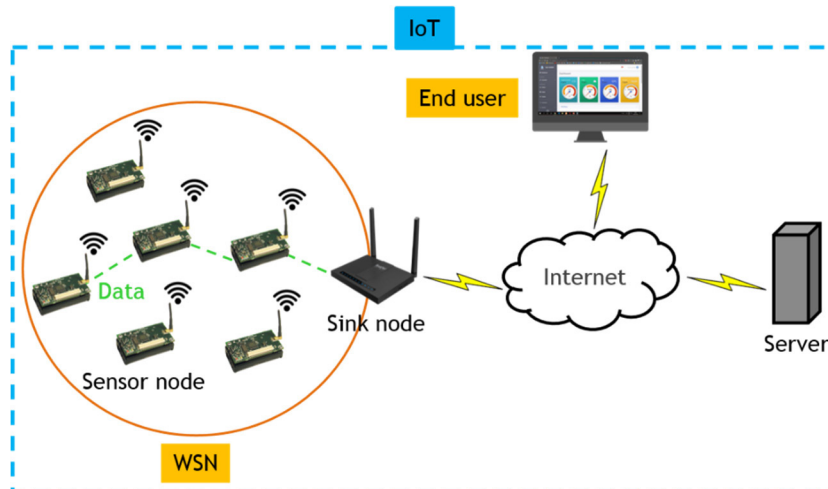


FIGURE 1. An IoT architecture integrating a WSN.

residual energy. This distributes traffic more evenly and helps extend network lifetime. However, EASDN and RL-based methods generate significantly more control traffic than shortest-path routing. They require frequent interactions with the WSN to select relay nodes and rely on broadcast control packets, which can congest the network and cause collisions with both control and data traffic. This leads to packet loss and accelerates the depletion of nodes' batteries, leaving less energy for sensing and data forwarding. As a result, data throughput decreases, and fewer packets reach end users, reducing the quality of the monitored information.

Additionally, RL-based routing algorithms, as well as those relying on meta-heuristics [12], [13], [14], [15], [16] and genetic algorithms [17], [18], impose high computational overhead on the controller and exhibit slow convergence [19], particularly in large-scale WSNs. As the network grows, these methods require more time to identify and disseminate the best relay node for each sensor node. Consequently, each sensor node continues to use its previously assigned relay for longer periods, leading to accelerated battery depletion and a shorter overall network lifetime.

In a previous study [20], we introduced NB-SDWSN, the first routing mechanism for Software-Defined Wireless Sensor Networks (SDWSNs) that leverages the Naïve Bayes algorithm, widely recognized for its effectiveness in classification tasks within Machine Learning (ML) [21]. The algorithm is grounded in Bayes' theorem, a well-established probabilistic model also known as the conditional probability theorem. Owing to its lightweight nature and computational efficiency [22], the Naïve Bayes-based approach enables the SDN controller to make routing decisions more rapidly than conventional algorithms, thereby improving network responsiveness and energy efficiency.

NB-SDWSN achieved notable improvements in overall network performance [20]. However, similar to most existing routing algorithms developed for SDWSNs, it did not consider two critical parameters—node buffer occupancy and

wireless link quality—during the routing process, i.e., during the selection of the most suitable neighboring node, for each sensor node, to serve as its relay. This limitation is particularly significant in WSNs, which are characterized by inherently low link quality and limited node buffer capacity as specified in the IEEE 802.15.4 standard [23]. Neglecting these parameters may lead to the selection of relay nodes with degraded link quality or an overloaded buffer, thereby increasing the probability of congestion, packet loss, and elevated network latency.

Motivated by the aforementioned observations, this paper proposes an enhanced congestion- and energy-aware routing approach based on the Naïve Bayes algorithm to improve the controller's routing decisions in SDWSNs. In contrast to NB-SDWSN, the proposed approach selects the best relay node for each node from its list of neighbors, taking into account multiple features, including the neighbor's battery level, its distance to the sink, its buffer occupancy, and the link quality that connects it to the sensor node. This multi-criteria selection process mitigates congestion, reduces network latency, and minimizes packet loss. Another key aspect of our proposed routing algorithm is that it periodically rotates the role of the relay node among the sensor node's neighbors, as it is based on time-varying features, such as the battery level. This fosters equitable traffic distribution among neighbors, resulting in balanced energy consumption and significantly extending the overall network lifespan.

In addition to our main contribution, our study makes several additional research contributions, summarized as follows:

- We present a comprehensive review of the routing protocols proposed for SDWSNs, analyzing their strengths, weaknesses, and key design factors such as load balancing, energy efficiency, and congestion avoidance. In contrast to prior surveys that focus mainly on algorithms sharing a similar concept to the proposed

method, our review spans a broad spectrum of routing approaches based on various design principles. Furthermore, we examine these algorithms based on often-overlooked factors in the literature, such as the volume of control traffic they generate within the WSN. This thorough review offers researchers and practitioners well-informed guidance for selecting and optimizing routing protocols for SDWSNs across diverse scenarios and traffic conditions.

- We undertake one of the first investigations into the effectiveness of routing algorithms based on Naïve Bayes theory for SDWSNs. Our primary contribution lies in the design and implementation of a novel routing algorithm within a framework that combines SDN with WSNs. To this end, we selected Software Defined Networking in Wireless Sensor nEtworks (SDN-WISE) [3], one of the most widely used platforms for SDN-based WSNs. We introduce minimal modifications to the original SDN-WISE data structures and packet formats to handle the fundamental operational principles of our proposed routing algorithm.
- While most routing algorithms have been evaluated only in small-scale WSNs, this study examines the performance of existing approaches—along with the proposed algorithm—in small as well as large-scale WSN deployments.
- We perform extensive simulations and use several metrics to evaluate and compare the performance of our proposed routing algorithm with others based on diverse design paradigms. This contrasts with most prior studies, which typically benchmark their algorithms only against methods built on similar underlying concepts.
- Nearly all research works proposing routing algorithms for SDWSNs have omitted evaluating the amount of control traffic generated by their approaches in WSNs. Acknowledging the critical impact of control traffic on overall network performance, we conduct a thorough assessment and comparison of the control traffic introduced by our proposed routing algorithm relative to existing solutions.

The remainder of the paper is structured as follows. Section II introduces the SDN paradigm in the context of WSNs. Section III presents a comprehensive review of the routing algorithms designed for controllers in SDWSNs. Section IV provides an overview of the Naïve Bayes theory, the probabilistic framework employed by the proposed routing algorithm to make routing decisions under uncertainty. Section V describes the design and operations of the proposed routing algorithm. Section VI outlines the simulation model, system parameters, and metrics used in our performance evaluation. It then presents a performance evaluation of our routing algorithm, comparing it with the well-established shortest path, EASDN, NB-SDWSN, and RL-based routing algorithms. Finally, Section VII concludes the paper and proposes potential avenues for future research.

TABLE 2. An example of a flow table.

Flow table of sensor node 5			
Packet type	Action	Relay node	Counter
Data	Forward	Node 3	2
Control	Forward	Node 3	1

II. SOFTWARE-DEFINED WIRELESS SENSOR NETWORKS (SDWSNs)

SDWSNs consist of two main components: the data plane and the control plane, as shown in Fig. 2. The data plane encompasses the WSN, consisting of sensor nodes responsible for collecting and forwarding data to the sink, which then transmits the received data via the Internet to a remote server, allowing the end user to access it. The control plane, on the other hand, houses the centralized controller. Both planes communicate with each other using control packets defined by the OpenFlow protocol [24]. Unlike traditional WSNs, where nodes independently determine routes to the sink, SDWSNs leverage the control plane to proactively compute and communicate, for each node, its path to the sink. This approach alleviates sensor nodes from the burden of the routing process, conserving their energy and consequently extending the network's lifetime.

A. DATA PLANE

The data plane, which comprises the sensor nodes, is responsible for executing packet processing and forwarding based on locally stored rules. Each node in the WSN maintains a flow table (routing table) in its memory, as shown in Table 2.

This table contains multiple pre-installed entries, referred to as flow or routing rules. A flow rule consists of four fields: packet type, action, relay node, and counter. The field packet type specifies the type of packet (control or data) that the routing rule handles. The action field defines the operation (forward, drop, or modify) to be performed on a packet when its type matches that of the routing rule. If the forward action is specified in the routing rule, the packet is sent through the neighbor indicated in the relay node field. This field is periodically updated as the controller regularly selects the best neighbor to act as the relay node for the sensor node. The counter field collects statistics, such as the number of times a packet matches the flow rule or the number of times the flow rule has been updated.

When a node generates or receives a packet, it first determines the packet type and checks its flow table for a rule specifying how to handle that type. If a match is found, the action defined in the routing rule is applied to the packet, and the flow rule's counter is incremented by one. In rare cases where no match exists (e.g., a new traffic type), the node requests an updated flow rule from the controller via the sink. The controller responds by sending a new flow rule, which is

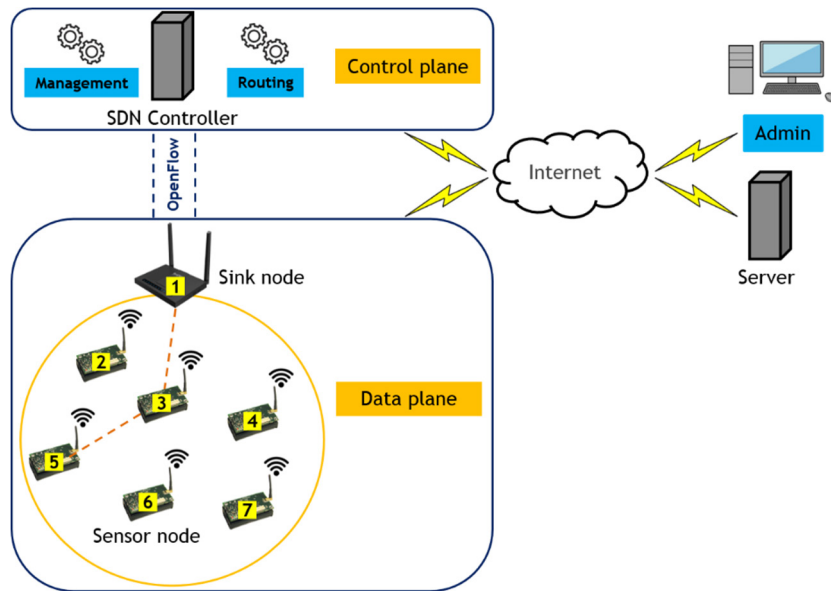


FIGURE 2. The SDWSN system architecture.

stored in the node's flow table to handle future packets of that type.

B. CONTROL PLANE

The control plane is composed of a centralized controller that oversees the network and makes high-level routing decisions. The controller, unlike the sensor nodes, is a resource-rich device with no energy constraints and can efficiently manage the complex routing task. To this end, the controller periodically receives status updates from the sensor nodes via the sink. These updates include vital metrics such as the distance to the sink and the battery level. Based on this information, the controller executes a routing algorithm to determine the best neighbor for each node to serve as its relay for forwarding packets toward the sink. For example, when the controller runs a shortest path routing algorithm, it focuses exclusively on the distance to the sink. It then selects the most suitable neighbor as the relay node for each node, specifically the one that requires the fewest hops to reach the sink. A routing table, mapping each node to its selected relay node, is then disseminated by the controller to the sink and subsequently to the sensor nodes. Each node extracts and learns its assigned relay node from the routing table and updates its pre-installed flow rules accordingly. This approach ensures that nodes always have up-to-date routing information, enhancing network efficiency.

In addition to offloading and relieving sensor nodes from the resource-heavy routing tasks, SDWSNs offer significant advantages by simplifying node management and programming. Traditional WSNs are challenging to manage, as administrators must manually configure each node to perform specific tasks. This process becomes increasingly tedious in large-scale WSNs with numerous nodes. By inte-

grating a centralized SDN controller, accessible via a network (e.g., the Internet), into WSNs, administrators can remotely manage sensor nodes by writing configuration programs at the controller, which then transmits them directly to the nodes, eliminating the need for physical access to each node.

Another drawback of traditional WSNs that SDN can overcome is their weak security. In conventional WSNs, data is typically transmitted without encryption, allowing unauthorized devices to access the network and communicate with sensor nodes. This exposes the network to attacks, including the interception and modification of packets. SDN mitigates these risks by enforcing security policies at the controller level, enabling end-to-end encryption, and restricting communication to authorized devices only [25], [26].

In the following section, we present a comprehensive review of existing studies in the literature that have explored the SDWSN concept introduced above.

III. RELATED WORK

This section provides a comprehensive review of existing routing solutions developed specifically for SDWSNs, with a focus on approaches that leverage the centralized control paradigm to optimize routing performance and energy efficiency. Note that the acronyms used in this section, as well as those used throughout the manuscript, are listed in Table 1.

Several studies [3], [4], [5] have proposed shortest-path routing algorithms to be executed by the controller. In this approach, each sensor node consistently uses the neighbor closest to the sink as its relay. This ensures that data packets traverse the minimum number of intermediate nodes, thereby minimizing network latency. However, this strategy leads to traffic concentration on a single relay node, resulting in rapid energy depletion of that node. Consequently, while

shortest-path routing improves latency, it adversely affects energy balancing and reduces the overall network lifetime.

The authors of [6] proposed EASDN, an opportunistic routing approach that periodically updates routing paths by selecting relay nodes based on their distance to the sink and residual energy. Specifically, the controller identifies the neighbors of a sensor node that are closest to the sink and selects the one with the highest battery level to serve as its relay. A routing table is then constructed and disseminated via control packets through the sink to the entire network. This periodic reassignment of relay nodes enables dynamic routing and contributes to extending the network lifetime by avoiding fixed paths. However, the primary reliance on proximity-based neighbor selection limits the potential for effective load balancing, especially in cases where a sensor node has only one suitable neighbor, reducing the algorithm's behavior to that of shortest-path routing. Furthermore, EASDN incurs substantial control overhead due to the broadcast of routing tables for all nodes, leading to increased energy consumption, processing delays, and network congestion. The resulting degradation in efficiency and rise in latency ultimately counteract the intended improvements in network longevity and performance.

The researchers in [27] introduced another opportunistic routing algorithm where the controller selects optimal relay nodes to the sink, considering factors such as transmission distance, residual energy, and hop count. The proposed approach manages to extend the network lifespan by implementing a rotational relay node strategy among a given sensor node's neighbors, thereby ensuring load balance. Nevertheless, a limitation of this approach is its reliance on the Euclidean distance for transmission calculations, necessitating the knowledge of actual sensor node coordinates. This requirement for geolocation equipment leads to increased energy consumption, which can be a drawback in practical deployments of SDWSNs, which are often composed of constrained devices characterized by limited battery, computation, storage, and communication resources.

The work in [12] presents a clustering-based routing algorithm for SDN-based WSNs that leverages the Whale Optimization Algorithm (WOA) [28], a metaheuristic optimization technique. The algorithm periodically organizes the network into clusters, each managed by a cluster head selected using a fitness function that considers the battery level, local node density, and inter-node distances. The cluster heads act as relay nodes, forwarding data and control packets from their respective cluster members to the sink. This approach promotes energy efficiency and extends the network lifetime by rotating the cluster head role among nodes, thereby distributing the communication load. Despite these benefits, the approach suffers from high computational complexity due to its hybrid design, which combines WOA with genetic mutation operations. This complexity limits scalability, particularly in large-scale WSNs. Moreover, the repeated cluster formation process generates considerable

control overhead, increasing communication between the data and control planes, which may introduce delays and degrade network performance.

The study in [15] proposed five clustering-based routing algorithms for SDN-based WSNs with mixed mobility, incorporating both stationary and mobile nodes. These algorithms use five meta-heuristic techniques—Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Imperialist Competitive Algorithm (ICA), Grey Wolf Optimizer (GWO), and WOA [29]—and rely on three fitness functions to optimize clustering and routing. The first determines the optimal number of clusters, the second selects cluster heads based on residual energy and proximity (favoring stationary nodes for stability), and the third identifies energy-efficient paths from cluster heads to the sink. To reduce control traffic, the algorithms introduce a loyalty mechanism that stabilizes cluster membership for mobile nodes by assuming low mobility (0.5 m/s). This reduces cluster reformation and the related message exchanges. Overall, the algorithms help maintain balanced clusters, reduce packet loss by avoiding mobile cluster heads, and improve energy efficiency through periodic cluster-head rotation. However, they converge slowly, delaying routing decisions, and their dependence on low mobility limits their applicability in highly dynamic IoT environments.

In [30], the authors proposed a clustering-based routing algorithm for SDWSNs utilizing the Artificial Bee Colony (ABC) algorithm [31], a swarm intelligence optimization technique. Initially, sensor nodes with the highest battery levels are selected as cluster heads, and the remaining nodes are assigned to the nearest cluster head based on the Euclidean distance. Subsequently, a new cluster head is selected within each cluster by identifying the node closest to the cluster's geometric center, using an ABC-based fitness function that minimizes the sum of the distances to all other cluster members. Finally, the algorithm identifies the optimal path from each cluster head to the sink by evaluating all possible paths and selecting the one with the lowest energy consumption. Although this approach reduces latency by favoring geographically proximate nodes, it introduces several limitations. The repeated use of the same path depletes the energy of the intermediate nodes, thereby shortening the network lifetime. Furthermore, since cluster head selection prioritizes centrality over battery level, the same node is repeatedly chosen, leading to rapid energy exhaustion and potential cluster disconnection. This results in increased packet loss, reduced throughput, and degraded overall network performance. Additionally, the frequent communication between nodes and the SDN controller, combined with the computational demands of the ABC algorithm, introduces routing delays, particularly in large-scale WSN deployments.

The first fuzzy logic-based routing algorithm for SDWSNs, as proposed in [32], uses multiple node parameters—remaining energy, buffer occupancy, and number of neighbors—to inform routing decisions. The algorithm employs a set of IF-THEN rules to assess node suitability as

a relay, translating numerical parameter values into linguistic fuzzy values (e.g., low, medium, high) based on predefined thresholds. Each neighbor's parameters are evaluated using these rules, and a corresponding numerical cost is computed to determine the most suitable relay node. This approach enables periodic reassignment of relay roles among neighbors, promoting balanced traffic distribution, uniform energy consumption, and improved network lifetime. By incorporating buffer occupancy, the algorithm also helps prevent node overload and congestion, thereby reducing packet loss and latency. However, it does not consider the distance to the sink, potentially selecting distant relay nodes and increasing end-to-end latency. Moreover, as the WSN size increases, fuzzy computations and associated control traffic impose significant computational and communication overheads on the controller, delaying routing decisions. These delays can result in the prolonged use of the same relay node, leading to energy depletion and reduced network longevity.

Several studies [7], [8], [9], [10] have proposed routing algorithms for SDWSNs using RL, particularly the Q-learning algorithm [33]. These algorithms periodically update routing configurations through a two-phase process involving routing table generation and iterative learning. The controller first generates all possible combinations of routing tables, each mapping sensor nodes to relay nodes based on their neighborhood information. A routing table is then randomly selected and disseminated across the network via the sink. Sensor nodes use the assigned relay nodes to transmit data for a fixed duration, during which performance metrics—such as packet delivery success, delay, and residual energy—are collected and aggregated into a reward value. Based on this reward, the controller either retains the current routing configuration or initiates another iteration to select a new routing table. This periodic adjustment promotes load balancing by rotating the relay responsibilities among neighbors, resulting in more uniform energy consumption and extended network lifetime. However, the RL-based approach suffers from several limitations. During the learning phase, nodes continuously exchange state information with the controller, which in turn sends action updates (i.e., routing tables) and reward feedback. This process generates significant control traffic, leading to increased energy consumption at the nodes and contributing to network congestion, higher latency, and packet loss. Moreover, the convergence process is slow and scales poorly with network size, often requiring numerous iterations to identify an optimal routing configuration. In larger networks, the algorithm may fail to converge, causing certain nodes to be repeatedly selected as relays, which accelerates their energy depletion and reduces overall network performance.

The work of [34] presented a routing algorithm using Deep Reinforcement Learning (DRL) [35], which combines principles of RL and Deep Learning (DL) [36], specifically through the use of Convolutional Neural Networks (CNNs) [37]. This algorithm uses a trained CNN model based on a dataset comprising various WSN topologies to

identify energy-efficient routes, prioritizing nodes with high residual energy and avoiding those with low energy levels, thus extending the network lifespan. However, this proposed method might opt for longer routes compared to traditional shortest-path algorithms, potentially increasing the packet's journey to the sink in terms of hop counts. Additionally, the scalability of the model is limited by its training on a specific set of WSN topologies, leading to potentially inaccurate route predictions for unencountered network layouts.

The study in [38] suggested a routing algorithm designed to minimize end-to-end delay by leveraging ML techniques, specifically RL and Artificial Neural Networks (ANNs) [39]. This algorithm trains a model on a dataset to predict routing paths for sensor nodes in actual network environments, aiming to reduce delay and enhance throughput, which is crucial for real-time IoT applications. However, a notable limitation of this approach is its lack of consideration for the energy levels of the nodes in the predicted paths. This oversight may result in the selection of nodes with low residual energy, risking premature energy depletion and potentially diminishing the network lifespan.

In our previous work [20], we introduced a preliminary version of a routing algorithm based on Naïve Bayes theory. The initial approach considered only two node features—battery level and distance to the sink—for relay node selection. This study extends that work by incorporating two additional features: buffer occupancy and link quality. These features are critical because they enable the algorithm to avoid selecting relay nodes that are either overloaded or exhibit poor link quality, thereby mitigating network congestion, minimizing latency, and reducing packet loss. Furthermore, the literature review in the previous study of [20] was limited, as it covered only four studies that proposed routing algorithms for SDWSNs. The present study addresses this limitation by providing a comprehensive review of the existing routing algorithms for SDWSNs based on various underlying concepts. In addition, the evaluation in the earlier work was restricted to a comparison with a single routing protocol. In contrast, the present study evaluates the proposed algorithm against multiple routing protocols, each based on a different paradigm. Moreover, the previous study did not analyze the control overhead introduced within the WSN by the proposed routing protocol or the one used for comparison. The current work incorporates an analysis of the generated control traffic, recognizing its substantial impact on the overall WSN performance.

Table 3 provides a summary of the various routing algorithms proposed for SDWSNs, evaluating each based on the following key aspects:

- **Load balancing awareness:** assesses whether the routing algorithm ensures that sensor nodes evenly distribute traffic among neighboring nodes, promoting uniform energy consumption and extending network lifetime. Additionally, it determines whether the algorithm rotates

TABLE 3. A summary of the routing algorithms proposed for SDWSNs, compared using eight criteria.

Study	Approach	Load balancing awareness	Energy awareness	Congestion awareness	Latency awareness	Link quality awareness	Generated control traffic	Network size	Convergence time
[3]	Dijkstra's algorithm	No	No	No	Yes	No	Low	Small	Fast
[4]	Dijkstra's algorithm	No	No	No	Yes	No	Low	Small	Fast
[5]	Dijkstra's algorithm	No	No	No	Yes	No	Low	Small	Fast
[6]	Opportunistic routing	Yes	Yes	No	Yes	No	High	Small	Fast
[27]	Opportunistic routing	Yes	Yes	No	Yes	Yes	Low	Large	Fast
[12]	Clustering + Meta-heuristic algorithm (WOA)	Yes	Yes	No	No	No	High	Large	Slow
[15]	Clustering + Meta-heuristic algorithms (GA, PSO, ICA, GWO, WOA)	Yes	Yes	No	Yes	No	Low	Large	Slow
[30]	Clustering + Swarm intelligence algorithm (ABC)	No	Yes	No	No	No	High	Large	Slow
[32]	Fuzzy logic	Yes	Yes	Yes	No	No	High	Large	Slow
[7]	RL (Q-learning algorithm)	Yes	Yes	No	Yes	No	High	Small	Slow
[8]	RL (Q-learning algorithm)	Yes	Yes	No	Yes	No	High	Small	Slow
[9]	RL (Q-learning algorithm)	Yes	Yes	No	Yes	No	High	Small	Slow
[10]	RL (Q-learning algorithm)	Yes	Yes	No	Yes	No	High	Small	Slow
[34]	DRL (CNN + RL)	Yes	Yes	No	No	No	Low	Small	Fast
[38]	ML (RL + ANN)	No	No	No	Yes	No	Low	Small	Fast
[20]	Naïve Bayes algorithm	Yes	Yes	No	Yes	No	Low	Large	Fast

the role of the relay node among a sensor node's neighbors.

- **Energy awareness:** examines whether the routing algorithm considers the battery level when making a routing decision, specifically when selecting the most suitable neighbor to act as a relay node for a given sensor node.
- **Congestion awareness:** evaluates whether the routing algorithm considers the buffer occupancy during the routing process.
- **Latency awareness:** determines whether the routing algorithm considers the distance to the sink when choosing a relay node.
- **Link quality awareness:** indicates whether the routing algorithm takes the link quality into consideration in the routing decision.
- **Generated control traffic:** reflects the amount of control traffic generated by the routing algorithm within the WSN.
- **Network size:** defines the size of the WSN under which the algorithm is evaluated. As in [3] and [6], a WSN with more than 32 nodes is considered large, while a WSN with 32 nodes or fewer is considered small.
- **Convergence time:** indicates how quickly the routing algorithm determines the relay nodes for all sensor nodes.

Table 3 reveals that only a few routing approaches for SDWSNs effectively achieve load balancing and ensure uniform energy consumption across nodes to extend network lifespan while generating minimal control traffic within the

WSN. Minimizing communication overhead between the controller and sensor nodes is crucial, as excessive communication degrades network performance by increasing energy consumption, reducing longevity, overloading nodes, and causing congestion. These issues lead to packet loss, delays at the nodes, and ultimately higher network latency. Moreover, most existing studies have not provided a comprehensive performance evaluation of their routing algorithms for both small- and large-scale network deployments. Assessing a routing algorithm under various scenarios is vital, as it helps identify its limitations and determine the conditions under which it performs well and those under which it does not perform well. This, in turn, allows for a better understanding of the WSN applications for which the routing algorithm is most suitable.

Another important observation from Table 3 is that several studies have overlooked the buffer status of the intermediate nodes when making routing decisions. This omission increases the likelihood of the controller selecting paths that traverse overloaded or congested nodes, resulting in higher network latency, increased packet loss, and reduced overall throughput. Moreover, these studies often neglect link quality when determining routes to the sink, frequently producing paths with poor-quality links that further exacerbate packet loss. To illustrate the impact of considering these features on WSN performance, Table 4 is provided. Additionally, many existing routing protocols do not incorporate probabilistic models for real-time adaptation to dynamic network conditions, leading to suboptimal routing decisions that adversely affect network reliability, energy efficiency, and overall lifespan.

TABLE 4. Node features and their role in routing and impact on WSN performance.

Feature	Role and impact on WSN performance
Battery level	It guides the routing algorithm to avoid selecting relay nodes with low energy levels, preventing their remaining battery from depleting too quickly and prolonging the network's lifetime.
Buffer occupancy	It influences the routing algorithm to avoid assigning an overloaded relay node for a sensor node, preventing congestion and mitigating packet loss. Moreover, it ensures that the sensor node's packets do not enter the queue of an overloaded relay node, where they would face long waiting times before being processed, thereby reducing overall network latency.
Distance to the sink	It guides the routing algorithm to elect a relay node close to the sink for the sensor node, ensuring the sensor node's packets are delivered to the sink with low latency.
Link quality	It directs the routing algorithm to designate a relay node close to the sensor node, reducing overall network latency while ensuring the successful delivery of the sensor node's packets and minimizing packet loss.

Motivated by the aforementioned observations, this paper presents a novel routing algorithm for SDWSNs based on Naïve Bayes theory, which leverages probabilistic inference capabilities well-suited for classification tasks under uncertain conditions. Unlike the existing approaches discussed in the literature review, the proposed algorithm aims to minimize communication overhead, prevent network congestion, improve overall throughput, and reduce network latency and packet loss by incorporating features often omitted in previous studies, namely link quality and node buffer occupancy. Moreover, it enhances the overall network lifespan through effective load balancing.

Owing to the lightweight and computationally efficient characteristics of the Naïve Bayes algorithm, the proposed algorithm provides a practical solution for real-time decision-making, thereby enhancing resource allocation and ensuring reliable data transmission within SDWSNs. By integrating a probabilistic approach, the proposed algorithm not only mitigates the inefficiencies of traditional routing techniques but also improves network performance by dynamically optimizing routing paths in response to changing network conditions. To better understand the underlying principles that enable such adaptability and efficiency, the following section presents the theoretical foundation of the Naïve Bayes algorithm, detailing its probabilistic model and relevance to the proposed routing mechanism.

IV. NAÏVE BAYES THEORY

In this study, the Naïve Bayes formulation is employed not as a data-driven ML classifier but as an analytical probabilistic inference model. The likelihood and prior probabilities are expressed using mathematical formulations derived from node features rather than from empirical training data. This

approach enables the computation of posterior probabilities in closed form, thereby avoiding iterative optimization or model training. Such analytical efficiency is particularly advantageous in SDWSNs, where nodes operate under severe constraints in terms of computation, communication, memory, and energy. Compared with alternative approaches such as linear regression, which relies on numerical optimization and assumes linear relationships, or decision tree algorithms, which require recursive partitioning and storage of hierarchical structures, the analytical Naïve Bayes model achieves faster inference with minimal overhead [40]. More complex methods like support vector machines and deep learning models demand substantial computational resources and large training datasets, making them impractical in constrained environments [41]. Similarly, fuzzy logic systems require extensive rule definition and parameter tuning, which increases implementation complexity [42]. Therefore, the analytical Naïve Bayes model provides an effective trade-off between accuracy, computational efficiency, and scalability, enabling real-time probabilistic decision-making in resource-constrained SDWSNs.

The Naïve Bayes algorithm is a probabilistic reasoning model widely used for classification tasks across various domains, including medical diagnosis, spam filtering, and decision-making under uncertainty [43]. Let $c_1, c_2, c_3, \dots, c_i$ denote the i possible classes. The objective of the Naïve Bayes algorithm is to determine the most probable and appropriate class c_i for a given instance, based on a set of observed features $X = \{x_1, x_2, x_3, \dots, x_j\}$. This classification problem is addressed using Bayes' theorem [44], also known as the conditional probability theorem, which provides a formal mathematical framework for computing the probability $P(c_i|X)$, i.e., the probability of class c_i given the observed features X . Assuming the conditional independence of the features $x_1, x_2, x_3, \dots, x_j$ given the class c_i , the probability $P(c_i|X)$ is computed as follows:

$$P(c_i|X) = P(c_i|x_1) \times P(c_i|x_2) \times \dots \times P(c_i|x_j) \quad (1)$$

The conditional probability $P(c_i|x_j)$ is calculated as follows:

$$P(c_i|x_j) = \frac{P(x_j|c_i) \times P(c_i)}{P(x_j)} \quad (2)$$

- $P(c_i|x_j)$ is the posterior probability, which indicates how likely the class c_i is to be selected given the observed feature x_j .
- $P(x_j|c_i)$ is the likelihood of observing the feature x_j given that the class c_i has been selected.
- $P(c_i)$ is the prior probability of class c_i , independent of any observed features.
- $P(x_j)$ is the marginal probability of observing the feature x_j .

In the context of routing in SDWSNs, we employ the Naïve Bayes algorithm to select the most appropriate neighbor (class) from a set of candidate neighbors (classes) to serve as a relay node for a given sensor node in forwarding packets

toward the sink. This selection process involves computing the posterior probability of each neighbor being chosen as a relay node given its features, namely battery level, distance to the sink, buffer occupancy, and link quality. These features serve as prior knowledge in the computation. The neighbor with the highest posterior probability is then assigned as the best relay node for the sensor node. For example, the probability of neighbor N_1 being designated as a relay node for sensor node S_3 , given N_1 's features, is expressed as the posterior probability: $P(N_1 | \text{battery level}_{N_1}, \text{distance to the sink}_{N_1}, \text{buffer occupancy}_{N_1}, \text{link quality}_{N_1, S_3})$. The detailed computation of this probability and the integration of the Naïve Bayes algorithm into the proposed routing framework are described in the following section.

V. PROPOSED ROUTING ALGORITHM

Our routing algorithm, which we denote as Naïve Bayes-based routing for Software-Defined Networking (NBSDN), has been implemented within the SDN-WISE framework [3]. In this section, we present a comprehensive overview of the processes and operations performed by the nodes (data plane) and the controller (control plane) in SDN-WISE. We also provide a detailed description of our routing algorithm and its two phases: (1) selecting the best neighbor for each sensor node to serve as its relay node using the Naïve Bayes algorithm with multiple features, including the battery level, and (2) utilizing the output of the first phase to construct and transmit routing paths to the sensor nodes, enabling them to learn their relay nodes for forwarding packets toward the sink. Lastly, in this section, a step-by-step example of the routing algorithm operating in a WSN with four nodes is provided to clearly highlight the way our approach works for better understanding.

For reference, Table 5 provides a comprehensive list of the symbols used throughout this study, ordered alphabetically.

A. DATA PLANE

1) DISCOVERING NEIGHBORING NODES

Each node, whether it is a sensor or sink node, creates and sends a beacon packet [3] every $T_{\text{beacon packet}}$ seconds to each node within its communication range. In this beacon packet, the node includes its address, its distance to the sink, and the address of the sink. The representation of a node's address (sensor or sink node) is presented below in "Address of the node". Note that in the beacon packets generated by the sink, it sets its own address in the field designated for the sink's address and sets 0 in the field designated for the distance to the sink.

When node A receives a beacon packet from node B , it performs the following tasks:

- It checks whether node B 's address is already listed in its neighbor table. If not, it adds node B 's address to the table.
- It acquires the sink's address from the beacon packet and stores it in its memory.

TABLE 5. List of symbols.

Symbol	Description
$Best\ relay\ node_{S_i}$	Neighboring node designated by NBSDN as the relay node for S_i
BL_{N_j}	Battery level of N_j
bl_{N_j}	Normalized battery level of N_j
BO_{N_j}	Buffer occupancy of N_j
bo_{N_j}	Normalized buffer occupancy of N_j
DS_{N_j}	Distance of N_j from the sink
ds_{N_j}	Normalized distance of N_j from the sink
l	Number of neighbors of a given node
LQ_{N_j, S_i}	Quality of the link between N_j and S_i
lq_{N_j, S_i}	Normalized quality of the link between N_j and S_i
n	Number of nodes in the WSN
N	Number of times a given node has been selected as a relay node by NBSDN
$NBSDN\ output$	Set of pairs mapping each sensor node to its relay node, designated by NBSDN
$Neighboring\ nodes_{S_i}$	List of neighboring nodes of S_i
N_j	Neighboring node of S_i
$Probabilities$	Array containing the probability of each neighbor of S_i being its relay node, computed by NBSDN
$Routing\ paths$	Array of arrays, each representing a routing path from a given node to the sink, constructed by NBSDN
S_i	Sink
$Selection\ history$	Set of tuples mapping each node to the number of times it has been selected as a relay node by NBSDN
$Sensor\ nodes$	List of sensor nodes in the WSN
S_i	Node i
$T_{\text{beacon packet}}$	Time interval separating the broadcast of two consecutive beacon packets
$T_{\text{controller routing}}$	Time interval between two consecutive executions of the routing algorithm by the controller
$T_{\text{data packet}}$	Time interval between the transmission of two consecutive data packets
T_{now}	Current simulation time
$T_{\text{report packet}}$	Time interval separating the transmission of two consecutive report packets

- It verifies whether node B 's distance to the sink, as indicated in the received beacon packet, is less than its distance to the sink, which is saved in its memory. If true, it updates the stored distance to node B 's distance plus one. It also updates a variable kept in its memory that contains its closest neighboring node to the sink, setting it to node B 's address. Node A utilizes the neighbor stored in this variable as its relay node to send its data and control packets to the sink during the period 0

$\leq T_{now} < T_{controller\ routing}$, where T_{now} denotes the actual time, and $T_{controller\ routing}$ represents the time at which the controller begins performing routing operations. This behavior occurs because, during this initial period, the controller has not yet started performing routing, as it is in the process of collecting information about the WSN topology. However, from $T_{now} \geq T_{controller\ routing}$, node A consistently obtains its relay node from the controller and uses it to transmit its data and control packets.

At the end of this process, every node in the WSN has its neighbor table populated with neighboring nodes within its communication range. It has also learned the address of the sink and its distance from it.

2) REPORTING NODE'S NEIGHBORS AND FEATURES TO THE CONTROLLER

Every $T_{report\ packet}$ seconds, each node within the WSN generates and transmits a report packet [3] to the sink via its best relay node that leads toward that node. The sink then forwards the received report packets to the controller. It is important to note that each $T_{report\ packet}$ seconds, the sink also creates its own report packet. However, unlike the sensor nodes, it sends this packet directly to the controller without passing it through intermediate nodes, as it has a direct link with the controller.

Within a report packet, a given node enumerates the list of its adjacent nodes identified in the phase “*discovering neighboring nodes*”. Thus, by periodically receiving all the report packets transmitted by the nodes in the WSN, the controller can establish and maintain an up-to-date representation of the WSN's topology. Additionally, the nodes furnish in the report packets details such as their residual energy, buffer occupancy, distance to the sink, and the quality of links with neighboring nodes. These features are critical for the proposed routing algorithm, which leverages them to identify the most suitable neighboring node for each sensor node to serve as its relay node toward the sink. The aforementioned fields are included in the report packet. It is worth noting that these fields were already present in the report packet of the original SDN-WISE implementation, except for the “*Buffer occupancy of the node*” field, which we have introduced.

a: ADDRESS OF THE NODE

The node's address is represented using 2 bytes and helps in differentiating and distinguishing the node from the others within the WSN. The first byte serves as the address prefix and is always set to 0. The second byte is an integer representing the node's identifier, which is assigned to the node at the startup of the WSN. This second byte ensures the uniqueness of the node's address, as each node in the WSN has a distinct identifier.

b: ADDRESSES OF THE NODE'S NEIGHBORING NODES

This field has a size of $2 \times l$ bytes, where l indicates the number of the node's neighbors. Each 2-byte segment of this

field specifies the address of a neighboring node, retrieved from the node's neighbors list, which was populated during the phase “*discovering neighboring nodes*”. Note that the address of a node's neighbor is represented in its dedicated 2-byte segment in the same manner as described earlier in the “*Address of the node*” field.

c: BATTERY LEVEL OF THE NODE

The battery level field is 1 byte in size and holds an integer value ranging from 0 to 255, where 0 indicates a fully depleted battery and 255 represents a fully charged state. This field serves as a crucial indicator of the node's remaining battery capacity.

As stated in [3], upon activation, each sensor node's battery has a maximum capacity of 5000 milliCoulombs (mC), and its energy consumption results from three primary activities: baseline operation, packet transmission, and packet reception. Equations (3) to (5) represent the energy consumption model adopted in this study, as defined in [3], which quantifies the reductions in the node's battery level resulting from these activities. As shown in equation (3), the sensor node consumes 6.8 mC per second to perform baseline tasks, including sensing physical quantities from the surrounding environment, storing and processing collected sensory data and received packets from neighboring nodes, and forming packets. Additionally, sending a 1-byte packet consumes 0.0027 mC, while receiving a 1-byte packet requires 0.00094 mC. Consequently, transmitting a packet of size L bytes depletes the sensor node's battery by $0.0027 \times L$ mC, as described in equation (4). Likewise, receiving a packet of length L bytes reduces the battery level by $0.00094 \times L$ mC, as specified in equation (5).

$$\text{Battery level} = \text{Battery level} - 6.8 \quad (3)$$

$$\text{Battery level} = \text{Battery level} - 0.0027 \times L \quad (4)$$

$$\text{Battery level} = \text{Battery level} - 0.00094 \times L \quad (5)$$

Initially, the battery level is a double-precision value, requiring an 8-byte field in the report packet. However, this significantly increases the length of the report packet, leading to higher energy consumption for both transmission and reception, as indicated in equations (4) and (5). To address this issue, the nodes in SDN-WISE convert and proportionally scale the battery level from its original decimal range of 0 to 5000 into an integer (byte) value within the range of 0 to 255 using equation (6). As a result, the battery level now occupies only one byte in the report packet, reducing its overall length.

$$\text{Battery level} = \text{Integer part of } \left(\frac{\text{Battery level}}{5000} \times 255 \right) \quad (6)$$

In this equation, the battery level is normalized from the range [0, 5000] to a decimal value between 0 and 1, then scaled to the range [0, 255]. The obtained value is passed through a truncation function to remove the decimal portion, ensuring that the final value is an integer.

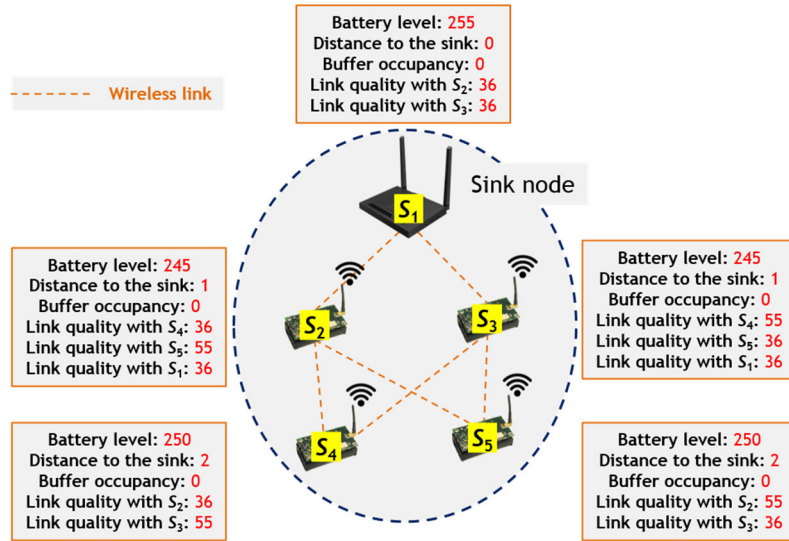


FIGURE 3. A WSN topology with 5 nodes built by the controller based on the received report packets.

Importantly, the sink, in contrast to the sensor nodes, is constantly plugged into an unlimited power source. Thus, it consistently reports a battery level of 255 in its report packets.

d: DISTANCE OF THE NODE FROM THE SINK

This 1-byte field stores an integer value, computed during the phase “*discovering neighboring nodes*”, indicating the number of hops required for a packet to traverse from a sensor node to the sink. It serves as a crucial metric, enabling the controller to assess the node’s proximity to the sink.

e: BUFFER OCCUPANCY OF THE NODE

This field occupies one byte in the report packet and contains an integer reflecting the number of packets currently stored in the node’s receiving queue. Incoming packets from neighboring nodes are temporarily buffered in this queue in the order of arrival before being processed and subsequently forwarded or dropped. The more packets a node’s receiving queue contains, the longer new incoming packets are delayed before being processed, increasing network latency.

The value of this field is a key indicator of the node’s workload. It ranges from 0 to 100 (the maximum capacity of the node’s receiving queue in SDN-WISE, measured in the number of packets), where 0 indicates an empty queue and 100 indicates a full queue, meaning the node is congested. Any additional received packets cannot be handled and must be dropped, increasing packet loss.

f: LINKS QUALITY WITH NEIGHBORING NODES

This field has a length of l bytes. The i th byte, where $i = 1, 2, \dots, l$, holds an integer value in the range $[0, 100]$, representing the quality of the wireless link between the node and its i th neighboring node in its neighbor table, as well as in the

field “*Addresses of the node’s neighboring nodes*”. A value of 0 represents the best link quality, while 100 represents the poorest. Each byte of this field functions as a metric indicating how far the neighbor is from the node. A lower byte value corresponds to a shorter distance between the node and the neighbor, and vice versa.

To determine the quality of the link with its i th neighbor, the node uses the Received Signal Strength Indicator (RSSI) [45], a widely used method for assessing the strength of a received signal in wireless communication systems. The node measures the power of the signal received from its i th neighbor. This measurement reflects factors such as distance, obstacles, and environmental conditions that affect signal propagation. The measured RSSI is represented as a negative value, where a higher value (i.e., closer to 0) signifies a stronger signal and a nearer neighbor to the node. The obtained RSSI value is then converted to a positive value using the absolute value function to fit within one byte, as illustrated in the equation below.

$$\text{Link quality } i\text{th neighbor} = |\text{RSSI } i\text{th neighbor}| \quad (7)$$

B. CONTROL PLANE

1) CONSTRUCTING THE WSN TOPOLOGY

Upon receiving a report packet from all the nodes in the WSN, the controller constructs the WSN topology and assigns each node its features based on its report packet. These features include battery level, distance to the sink, buffer occupancy, and the quality of links with neighboring nodes. An example of a WSN topology built by the controller is shown in Fig. 3.

2) EXECUTING THE NBSDN ROUTING ALGORITHM

Our proposed routing algorithm consists of two main phases. In the first phase, the controller identifies and selects, for each sensor node in the WSN, the best neighbor to act as its relay

node to forward its packets to the sink. In the second phase, the controller constructs routing paths based on the results of the first phase and then transmits them to the data plane (WSN), enabling the sensor nodes to learn their best relay node selected by the controller. Both phases are executed periodically every $T_{controller\ routing}$ seconds. Consequently, after each $T_{controller\ routing}$ second, a different neighboring node may serve as the relay node for a given sensor node. This mechanism enables a sensor node to evenly distribute its traffic among its neighbors, leading to balanced energy consumption and an extended network lifetime.

Let us consider a WSN comprising n nodes: one sink node, referred to as S_1 , and a set of sensor nodes denoted as *Sensor nodes*, where *Sensor nodes* = $\{S_2, S_3, \dots, S_n\}$. Each node S_i , where $i = 1, 2, 3, \dots, n$, possesses a set of neighboring nodes labeled as *Neighboring nodes_{Si}*, where *Neighboring nodes_{Si}* = $\{N_1, N_2, N_3, \dots, N_l\}$, with l being the total count of the node's adjacent nodes, and *Neighboring nodes_{Si}* $\subset$ *Sensor nodes* $\cup S_1$. Node S_i has four features: battery level (BL_{Si}), distance to the sink (DS_{Si}), buffer occupancy (BO_{Si}), and the quality of the link with each of its neighbors ($LQ_{N1,Si}, LQ_{N2,Si}, LQ_{N3,Si}, \dots, LQ_{Nl,Si}$).

α : BEST RELAY NODE IDENTIFICATION

The first input to this phase is a complete WSN topology graph constructed by the controller during the phase “*constructing the WSN topology*”. The second input to this phase is a set of pairs $\langle S_i, N \rangle$, denoted as *Selection history*, where N represents the number of times node S_i has been selected as a relay node for other nodes by the controller. This set can be valuable if multiple neighboring nodes have the same probability of being selected as the best relay node for a given sensor node. In such cases, the controller resolves the tie by selecting the neighboring node that has been chosen least frequently as a relay for other nodes, ensuring balanced traffic distribution across the WSN and thereby extending the network lifetime. The output of the phase “*best relay node identification*” is a set of pairs $\langle S_i, \text{Best relay nodes}_{Si} \rangle$, denoted as *NBSDN output*, where $S_i \in \text{Sensor nodes}$ and *Best relay nodes_{Si}* $\in$ *Neighboring nodes_{Si}*.

For each node S_i ($S_i \in \text{Sensor nodes}$), the controller checks if it has the sink S_1 as a neighbor, i.e., $S_1 \in \text{Neighboring nodes}_{Si}$. If true, the controller immediately designates S_1 as its best relay node and adds the pair $\langle S_i, S_1 \rangle$ to the *NBSDN output* set. It also increments N by one in the pair $\langle S_1, N \rangle$ stored in the *Selection history* set. Otherwise, for each neighbor N_j of node S_i ($N_j \in \text{Neighboring nodes}_{Si}$), where $j = 1, 2, 3, \dots, l$, the controller performs the following steps 1 to 4:

STEP 1: Normalize the features of neighbor N_j .

The values of the features of N_j are normalized to a common scale range of 0 to 1 to prevent any single feature from exerting disproportionate influence due to its numerical range. This ensures balanced contributions from all features, eliminating bias toward those with larger values.

The value of the feature “battery level” (BL_{Nj}) is normalized using equation (8). The normalized battery level is labeled as bl_{Nj} .

$$bl_{Nj} = \frac{1}{255 - BL_{Nj}} \quad (8)$$

In the above equation, the expression $255 - BL_{Nj}$ computes the amount of battery consumed by the neighbor N_j . The obtained value is then inverted so that the neighbor that has consumed the least amount of its battery has the highest normalized battery level value and consequently the highest probability of being selected as the relay node for node S_i . Avoiding the selection of a neighbor with a low battery level as a relay node prevents its battery from draining completely, which would cause it to switch off. This maintains the neighboring node for an extended duration and extends the overall network lifespan.

The value of the feature “distance to the sink” (DS_{Nj}) is normalized using equation (9). The normalized value is denoted as ds_{Nj} .

$$ds_{Nj} = \frac{1}{DS_{Nj}} \quad (9)$$

In equation (9), the distance to the sink is inverted to ensure that the closest neighbor to the sink, which requires the minimum number of hops to deliver packets to the sink, has the highest normalized distance to the sink and, therefore, the highest likelihood of being chosen as the relay node for node S_i . This selection principle ensures that S_i 's packets reach the sink with lower delay, resulting in low network latency.

The value of the feature “buffer occupancy” (BO_{Nj}) is normalized using equation (10). The normalized buffer occupancy is represented as bo_{Nj} .

$$bo_{Nj} = \frac{1}{BO_{Nj}} \quad (10)$$

In this equation, the buffer occupancy value is inverted to guarantee that the neighbor with the fewest packets in its receiving queue has the highest normalized buffer occupancy value, thereby maximizing its probability of being designated as the relay node for node S_i . Choosing the neighbor with the lowest buffer occupancy as the relay node ensures that S_i 's packets experience minimal waiting time in the relay node's receiving queue before being processed and either forwarded or dropped, or no waiting time at all if the queue is empty, thereby reducing overall network latency. Moreover, this selection criterion avoids designating as relay nodes the neighbors that have a significant number of packets in their queue. This prevents overloading them, helps preserve their energy, and increases network lifetime. Additionally, it mitigates node congestion, thereby minimizing packet loss.

The feature “link quality” $LQ_{Nj,Si}$ is normalized using equation (11), and the resulting normalized value is expressed as $lq_{Nj,Si}$.

$$lq_{Nj,Si} = \frac{1}{LQ_{Nj,Si}} \quad (11)$$

In equation (11), the value of LQ_{N_j, S_i} is inverted so that the neighbor with the lowest link quality (RSSI) to S_i (i.e., the closest neighbor to the sensor node S_i) has the highest normalized link quality value, thus increasing its chance of being assigned as the relay node for S_i . This guarantees the reliable transmission of S_i 's packets to its chosen relay node, minimizing packet loss. Additionally, it ensures that S_i 's packets are transmitted with lower delay to the relay node, reducing overall network latency.

STEP 2: Compute the conditional probability $P(N_j | bl_{N_j}, ds_{N_j}, bo_{N_j}, lq_{N_j, S_i})$ and store the resulting value in an array denoted as *Probabilities*.

The expression $P(N_j | bl_{N_j}, ds_{N_j}, bo_{N_j}, lq_{N_j, S_i})$ quantifies the likelihood of selecting neighbor N_j as the best relay node for sensor node S_i , given N_j 's normalized features. Note that each feature is independent of the others and depends only on N_j , so this probability can be calculated as follows using equation (12).

$$P(N_j | bl_{N_j}, ds_{N_j}, bo_{N_j}, lq_{N_j, S_i}) = P(N_j | bl_{N_j}) \times P(N_j | ds_{N_j}) \times P(N_j | bo_{N_j}) \times P(N_j | lq_{N_j, S_i}) \quad (12)$$

The posterior probabilities $P(N_j | bl_{N_j})$, $P(N_j | ds_{N_j})$, $P(N_j | bo_{N_j})$, and $P(N_j | lq_{N_j, S_i})$ are determined using Bayes' theorem and are given by equations (13) to (16), respectively.

$$P(N_j | bl_{N_j}) = \frac{P(bl_{N_j} | N_j) \times P(N_j)}{P(bl_{N_j})} \quad (13)$$

$$P(N_j | ds_{N_j}) = \frac{P(ds_{N_j} | N_j) \times P(N_j)}{P(ds_{N_j})} \quad (14)$$

$$P(N_j | bo_{N_j}) = \frac{P(bo_{N_j} | N_j) \times P(N_j)}{P(bo_{N_j})} \quad (15)$$

$$P(N_j | lq_{N_j, S_i}) = \frac{P(lq_{N_j, S_i} | N_j) \times P(N_j)}{P(lq_{N_j, S_i})} \quad (16)$$

The prior probability $P(N_j)$ refers to the likelihood that, among the l neighbors of sensor node S_i , neighbor N_j is designated as its relay node. It is computed using equation (17).

$$P(N_j) = \frac{1}{l} \quad (17)$$

The likelihoods $P(bl_{N_j} | N_j)$, $P(ds_{N_j} | N_j)$, $P(bo_{N_j} | N_j)$, and $P(lq_{N_j, S_i} | N_j)$ represent the probabilities of observing the feature values bl_{N_j} , ds_{N_j} , bo_{N_j} , and lq_{N_j, S_i} , given that neighbor N_j has already been chosen as the relay node for sensor node S_i . These likelihoods quantify the relative contribution of each feature of N_j to the overall probability of choosing N_j as the best relay node for S_i . They are calculated using equations (18) to (21), respectively.

$$P(bl_{N_j} | N_j) = \frac{bl_{N_j}}{bl_{N_j} + ds_{N_j} + bo_{N_j} + lq_{N_j, S_i}} \quad (18)$$

$$P(ds_{N_j} | N_j) = \frac{ds_{N_j}}{bl_{N_j} + ds_{N_j} + bo_{N_j} + lq_{N_j, S_i}} \quad (19)$$

$$P(bo_{N_j} | N_j) = \frac{bo_{N_j}}{bl_{N_j} + ds_{N_j} + bo_{N_j} + lq_{N_j, S_i}} \quad (20)$$

$$P(lq_{N_j, S_i} | N_j) = \frac{lq_{N_j, S_i}}{bl_{N_j} + ds_{N_j} + bo_{N_j} + lq_{N_j, S_i}} \quad (21)$$

Each likelihood is computed as the ratio of a given feature to the sum of all features of N_j , ensuring that the total contribution of all features sums to 1. This normalization provides a relative measure of each feature's influence on the decision to select N_j as the relay node for S_i , while ensuring that no single feature unfairly dominates or biases the relay selection process.

Equations (22) to (25) define the marginal probabilities of the features bl_{N_j} , ds_{N_j} , bo_{N_j} , and lq_{N_j, S_i} for the neighbor N_j . These probabilities represent the relative contribution of each feature of N_j compared to the same feature among all neighbors of sensor node S_i .

$$P(bl_{N_j}) = \frac{bl_{N_j}}{\sum_{k=1}^l bl_{N_k}} \quad (22)$$

$$P(ds_{N_j}) = \frac{ds_{N_j}}{\sum_{k=1}^l ds_{N_k}} \quad (23)$$

$$P(bo_{N_j}) = \frac{bo_{N_j}}{\sum_{k=1}^l bo_{N_k}} \quad (24)$$

$$P(lq_{N_j, S_i}) = \frac{lq_{N_j, S_i}}{\sum_{k=1}^l lq_{N_k, S_i}} \quad (25)$$

Each marginal probability is computed as the ratio of a specific feature of N_j to the sum of the same feature across all neighbors of S_i . This normalization ensures that the influence of each feature is assessed in a comparative manner rather than in isolation, meaning it is evaluated through an inter-node comparison rather than an intra-node comparison. In this approach, the influence of a given feature is considered among all neighbors of S_i rather than being limited to node N_j alone, as in the likelihoods expressed in equations (18) to (21). This ensures a fair assessment of how N_j ranks as a potential relay node for S_i among its competing nodes, allowing the relay selection process to account for the relative strengths and weaknesses of all candidate neighbors.

Once the controller completes the execution of STEP 1 and STEP 2 on all neighboring nodes of S_i , it proceeds with the following STEPS 3 and 4:

STEP 3: Select the best relay node for S_i .

Before describing the operations performed in this step, it is important to highlight that the optimal neighbor, which has the highest normalized feature values for the battery level, distance to the sink, buffer occupancy, and link quality, yields the highest numerator in the posterior probabilities computed in (13) to (16), resulting in the lowest conditional probability $P(N_j | bl_{N_j}, ds_{N_j}, bo_{N_j}, lq_{N_j, S_i})$ among the candidate neighbors, and vice versa. Consequently, in the *Probabilities* array, we invert the value of the conditional probability $P(N_j | bl_{N_j}, ds_{N_j}, bo_{N_j}, lq_{N_j, S_i})$ (i.e., $1 / P(N_j | bl_{N_j}, ds_{N_j}, bo_{N_j}, lq_{N_j, S_i})$) for each neighbor to ensure that the neighbor with the best features (i.e., the one with the lowest $P(N_j | bl_{N_j}, ds_{N_j}, bo_{N_j}, lq_{N_j, S_i})$) is assigned the highest value, while the least optimal neighbor, with the worst features, is assigned the lowest value.

Algorithm 1 Relay Node Selection**Inputs:** WSN topology graph and *Selection history*: set of pairs $\langle S_i, N \rangle$ **Output:** *NBSDN output* consisting of a set of pairs $\langle S_i, \text{Best relay nodes}_i \rangle$

```

1  Initialize the NBSDN output set as empty
2  Initialize an array labeled Probabilities as empty
3  For each  $S_i$  ( $S_i \in \text{Sensor nodes}$ ) do
4      If  $S_1 \in \text{Neighboring nodes}_{S_i}$  then
5          Add the pair  $\langle S_i, S_1 \rangle$  to the NBSDN output set
6          Find the pair  $\langle S_1, N \rangle$  in the Selection history set and increment  $N$  by one
7      End If
8      Else
9          For each  $N_j$  ( $N_j \in \text{Neighboring nodes}_{S_i}$ ) do
10             Normalize  $BL_{N_j}$ ,  $DS_{N_j}$ ,  $BO_{N_j}$ , and  $LQ_{N_j, S_i}$  using equations (8) to (11), respectively, and store the resulting values
               in  $bl_{N_j}$ ,  $ds_{N_j}$ ,  $bo_{N_j}$ , and  $lq_{N_j, S_i}$ 
11             Calculate the conditional probability  $P(N_j | bl_{N_j}, ds_{N_j}, bo_{N_j}, lq_{N_j, S_i})$  using equation (12)
12             Invert the computed probability and store the result in the array Probabilities
13          End For each
14      End Else
15      Identify the neighbor(s) with the maximum value in the array Probabilities
16      If a single neighbor is identified (referred to as  $N_{max}$ ) then
17          Add the pair  $\langle S_i, N_{max} \rangle$  to the NBSDN output set
18          Find the pair  $\langle N_{max}, N \rangle$  in the Selection history set and increment  $N$  by one
19      End If
20      Else (multiple neighbors have the same maximum value)
21          Select the one (also denoted as  $N_{max}$ ) with the minimum value of  $N$  in the Selection history set
22          Add the pair  $\langle S_i, N_{max} \rangle$  to the NBSDN output set
23          Find the pair  $\langle N_{max}, N \rangle$  in the Selection history set and increment  $N$  by one
24      End Else
25  End For each

```

Subsequently, from the array *Probabilities*, the neighbor with the highest probability value is selected as the best relay node for sensor node S_i . If multiple neighbors share the same maximum value, the controller breaks the tie using the *Selection history* set. It chooses the neighbor that has been assigned as a relay node the fewest times for other nodes, i.e., the neighbor with the smallest N value in the *Selection history* set. Finally, a pair consisting of sensor node S_i and its designated relay node is added to the *NBSDN output* set.

STEP 4: Update the *Selection history* set.

In the *Selection history* set, for the pair corresponding to the node elected as the best relay node for S_i , the controller increments its N value by one.

Algorithm 1 presents a summary of the procedure executed by the controller in the phase “*best relay node identification*”. Once the controller finishes iterating through all the sensor nodes to find their best relay nodes, it feeds the *NBSDN output* set, pairing each sensor node with its selected relay node using the Naïve Bayes algorithm, to the next phase, “*routing paths construction and transmission*”.

b: ROUTING PATHS CONSTRUCTION AND TRANSMISSION

The input to this phase is the *NBSDN output* set, which serves as the basis for constructing a set of routing paths to the sink. Each path originates from the sink, traverses a series of

intermediate sensor nodes, and terminates at a specific sensor node. The output of this phase is a set of control packets [3], each containing a routing path. These packets are sent to the WSN, enabling sensor nodes to identify and learn their best relay nodes assigned by the controller and transmit both control and data packets to the sink through them. In this phase, the controller executes two steps:

STEP 1: Construct the routing paths toward the sink.

For each pair $\langle S_i, \text{Best relay nodes}_i \rangle$ in *NBSDN output*:

- If the relay node of the pair is the sink (i.e., $\text{Best relay nodes}_i = S_1$), create a new routing path that begins with the sink S_1 , followed by the sensor node S_i of the pair. The newly created routing path is $[S_1, S_i]$.
- If the relay node of the pair is not the sink (i.e., $\text{Best relay nodes}_i \neq S_1$), search in the previously created routing paths for one whose last element is equal to the relay node of the pair (i.e., a path $[S_1, \dots, \text{Best relay nodes}_i]$). If such a path is found, append the sensor node S_i of the pair to its end, resulting in the modified routing path $[S_1, \dots, \text{Best relay nodes}_i, S_i]$. Otherwise, from the routing paths created so far, search for and extract a sub-path that starts at the sink and ends at the relay node of the pair. Then, create a new routing path identical to this sub-path, with the sensor node S_i of the pair appended to the end.

Algorithm 2 Routing paths construction**Input:** *NBSDN output* consisting of a set of pairs $\langle S_i, \text{Best relay node}_{S_i} \rangle$ **Output:** *Routing paths*: array of arrays, each representing a routing path of the form $[S_1, \dots, S_i]$

```

1  Initialize the array Routing paths as empty
2  For each pair  $\langle S_i, \text{Best relay node}_{S_i} \rangle$  in the NBSDN output set do
3      If  $\text{Best relay node}_{S_i} = S_1$  then
4          Add a new path  $[S_1, S_i]$  to Routing paths
5      End If
6      Else
7          Search in Routing paths for a path structured as  $[S_1, \dots, \text{Best relay node}_{S_i}]$ 
8          If such a path is found then
9              Add  $S_i$  to the end of that path
10         End If
11         Else
12             Search in Routing paths for a path structured as  $[S_1, \dots, \text{Best relay node}_{S_i}, \dots, S_m]$ , where  $m \in \{2, 3, \dots, n\}$ 
13             Once such a path is found, extract from it the sub-path  $[S_1, \dots, \text{Best relay node}_{S_i}]$ 
14             Create a new path that is identical to the extracted sub-path
15             Add  $S_i$  to the end of the new path
16             Add the new path  $([S_1, \dots, \text{Best relay node}_{S_i}, S_i])$  to Routing paths
17         End Else
18     End Else
19 End For each

```

The resulting routing paths take the form of, for example, $[S_1, S_2, S_4, \dots, S_i]$. Every path is an array ordered in such a way that, for each sensor node in the array, its best relay node, as selected by the controller, is the preceding node. In our example, the best relay node for S_2 is S_1 , for S_4 is S_2 , and so on. A summary of the procedure for forming routing paths is provided in Algorithm 2.

STEP 2: Communicate the routing paths to the WSN.

Each constructed routing path, such as $[S_1, S_2, S_4, \dots, S_i]$, is encapsulated in a control packet and transmitted to the sink. Upon receiving the packet, the sink forwards it to the node that comes after it in the routing path, in this case, S_2 . However, before doing so, it removes itself from the encapsulated path (resulting in a shortened path $[S_2, S_4, \dots, S_i]$) to reduce packet length and minimize the energy consumption of the nodes traversed by the packet. When the sensor node S_2 receives the control packet, it recognizes the sink S_1 as its best relay node since it received the packet from it. S_2 then transmits the packet to the next node in the routing path, S_4 , once again removing itself from the path within the packet (which now becomes $[S_4, \dots, S_i]$) to further minimize packet length. S_4 receives the control packet and learns that S_2 is its best relay node, as the packet comes from it. This operation is repeated at each visited node until the packet reaches its final destination.

C. EXAMPLE

To illustrate the functionality of our proposed NBSDN algorithm, let us apply it to a simple WSN topology consisting of four nodes: one sink node and three sensor nodes, as depicted in Fig. 4. This WSN topology is constructed at

the controller level, based on control packets received from the WSN, shortly before a $T_{\text{controller routing}}$ interval elapses.

As soon as this interval elapses, the controller executes the first phase of the NBSDN algorithm, namely “*best relay node identification*”.

1) BEST RELAY NODE IDENTIFICATION

The controller examines each sensor node in the WSN, starting from those closest to the sink, to identify its best relay node. In the illustrated example, the controller sequentially examines nodes S_2 , S_3 , and S_4 to determine their respective best relay node.

a: S_2 AND S_3

Both of these nodes have the sink S_1 as a neighbor, so the routing algorithm directly assigns it as their best relay node and adds two pairs $\langle S_2, S_1 \rangle$ and $\langle S_3, S_1 \rangle$ to the *NBSDN output* set. Moreover, in the *Selection history* set, the N value of the pair corresponding to the sink S_1 is increased by two.

b: S_4

S_4 has nodes S_2 and S_3 as neighbors ($l = 2$). The controller determines which of the two nodes is the most suitable relay. To this end, it performs the following steps.

NEIGHBOR S_2 **STEP 1: Normalize the features of neighbor S_2 .**

The values of the features: battery level (BL_{S_2}), distance to the sink (DS_{S_2}), buffer occupancy (BO_{S_2}), and link quality (LQ_{S_2, S_4}) are normalized:

$$bl_{S_2} = \frac{1}{255 - BL_{S_2}} = \frac{1}{255 - 205} = \frac{1}{50} = 0.02$$

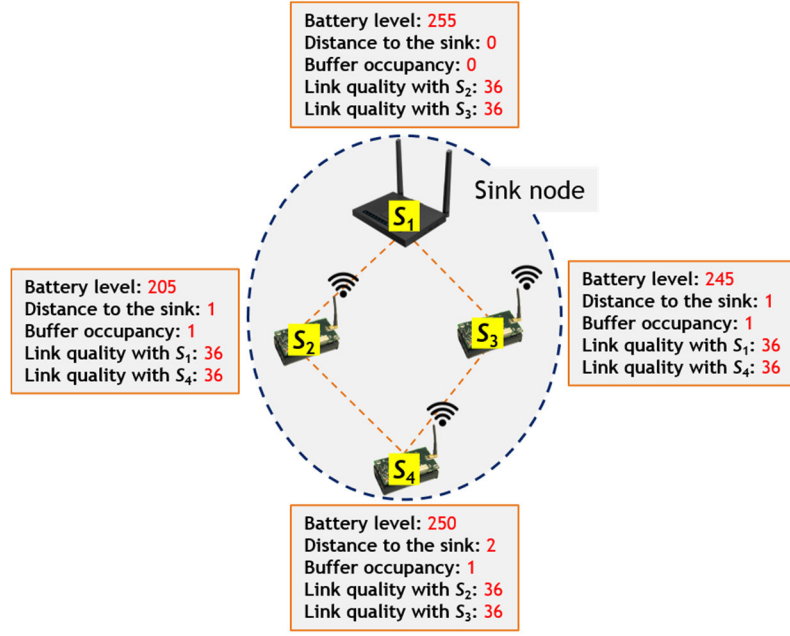


FIGURE 4. A WSN topology with four nodes and their corresponding feature values.

$$ds_{S2} = \frac{1}{DS_{S2}} = \frac{1}{1} = 1$$

$$bos_{S2} = \frac{1}{BO_{S2}} = \frac{1}{1} = 1$$

$$lq_{S2,S4} = \frac{1}{LQ_{S2,S4}} = \frac{1}{36} = 0.03$$

STEP 2: Compute the conditional probability $P(S_2 | bl_{S2}, ds_{S2}, bos_{S2}, lq_{S2,S4})$ and store the resulting value in an array denoted as *Probabilities*.

$$\begin{aligned}
 &P(S_2 | bl_{S2}, ds_{S2}, bos_{S2}, lq_{S2,S4}) \\
 &= P(S_2 | bl_{S2}) \times P(S_2 | ds_{S2}) \\
 &\times P(S_2 | bos_{S2}) \times P(S_2 | lq_{S2,S4}) \\
 &= \frac{P(bl_{S2} | S_2) \times P(S_2)}{P(bl_{S2})} \times \frac{P(ds_{S2} | S_2) \times P(S_2)}{P(ds_{S2})} \\
 &\times \frac{P(bos_{S2} | S_2) \times P(S_2)}{P(bos_{S2})} \times \frac{P(lq_{S2,S4} | S_2) \times P(S_2)}{P(lq_{S2,S4})} \\
 &= \frac{bl_{S2}}{bl_{S2} + ds_{S2} + bos_{S2} + lq_{S2,S4}} \times \frac{1}{I} \\
 &\times \frac{ds_{S2}}{bl_{S2} + ds_{S2} + bos_{S2} + lq_{S2,S4}} \times \frac{1}{I} \\
 &\times \frac{bos_{S2}}{bl_{S2} + ds_{S2} + bos_{S2} + lq_{S2,S4}} \times \frac{1}{I} \\
 &\times \frac{lq_{S2,S4}}{bl_{S2} + ds_{S2} + bos_{S2} + lq_{S2,S4}} \times \frac{1}{I} \\
 &\times \frac{lq_{S2,S4}}{lq_{S2,S4} + lq_{S3,S4}}
 \end{aligned}$$

$$\begin{aligned}
 &= \frac{0.02}{0.02+1+1+0.03} \times \frac{1}{2} \times \frac{1}{0.02+1+1+0.03} \times \frac{1}{2} \\
 &\times \frac{0.02}{0.02+0.1} \times \frac{1}{1+1} \times \frac{0.03}{0.02+1+1+0.03} \times \frac{1}{2} \\
 &\times \frac{1}{0.02+1+1+0.03} \times \frac{1}{2} \times \frac{0.03}{0.03+0.03} \\
 &P(S_2 | bl_{S2}, ds_{S2}, bos_{S2}, lq_{S2,S4}) \\
 &= 0.03 \times 0.49 \times 0.49 \times 0.01 = 7.2 \times 10^{-5}
 \end{aligned}$$

NEIGHBOR S3

Both STEP 1 and STEP 2 are subsequently applied to neighbor S3, yielding the conditional probability $P(S_3 | bl_{S3}, ds_{S3}, bos_{S3}, lq_{S3,S4}) = 6.62 \times 10^{-5}$. Following these steps, the controller proceeds to execute STEP 3 for both S2 and S3.

STEP 3: Select the best relay node for S4.

The conditional probabilities $P(S_2 | bl_{S2}, ds_{S2}, bos_{S2}, lq_{S2,S4})$ and $P(S_3 | bl_{S3}, ds_{S3}, bos_{S3}, lq_{S3,S4})$, stored in the array *Probabilities*, are inverted, resulting in the updated array: $[S_2: 1 / 7.2 \times 10^{-5}, S_3: 1 / 6.62 \times 10^{-5}]$. Based on these values, the controller selects S3 as the best relay node for S4 since it corresponds to the highest value in the *Probabilities* array. The controller then adds the pair $\langle S_4, S_3 \rangle$ to the *NBSDN output* set.

STEP 4: Update the Selection history set.

In the *Selection history* set, for the pair corresponding to node S3, the controller increments its *N* value by one.

At the end of the phase “best relay node identification”, the *NBSDN output* set contains the following pairs: $\langle S_2, S_1 \rangle$, $\langle S_3, S_1 \rangle$, and $\langle S_4, S_3 \rangle$. The controller then feeds this set to the subsequent phase, “routing paths construction and transmission”.

2) ROUTING PATHS CONSTRUCTION AND TRANSMISSION

STEP 1: Construct the routing paths toward the sink.

The relay node in the pair $\langle S_2, S_1 \rangle$ is the sink S_1 . As a consequence, the controller creates a new routing path starting with S_1 , followed by the sensor node to which the controller assigned S_1 as its relay node (i.e., S_2), resulting in the path $[S_1, S_2]$. Similarly, for the pair $\langle S_3, S_1 \rangle$, where S_1 is also the relay node, the controller applies the same operation, yielding the routing path $[S_1, S_3]$.

In the third pair $\langle S_4, S_3 \rangle$, the sink S_1 is not the relay node; instead, it is the sensor node S_3 . Thus, the controller searches through the routing paths created so far for a path ending with S_3 . It finds the path $[S_1, S_3]$ and appends S_4 (the sensor node for which the controller designated S_3 as its relay node) to its end. The routing path becomes: $[S_1, S_3, S_4]$.

STEP 2: Communicate the routing paths to the WSN.

The controller transmits the constructed routing paths $[S_1, S_2]$ and $[S_1, S_3, S_4]$ to the sink using control packets. For the first control packet encapsulating the routing path $[S_1, S_2]$, the sink forwards it to the sensor node that comes after it in the path, S_2 . Before forwarding, it removes itself from the path, leaving $[S_2]$ as the updated path within the packet. Upon receiving the packet, S_2 identifies S_1 as its best relay node, since it received the control packet from that node.

For the second control packet containing the routing path $[S_1, S_3, S_4]$, the sink sends it to the sensor node following it in the path, namely S_3 . Before sending the packet, the sink removes itself from the path encapsulated within the packet, updating it to $[S_3, S_4]$. When S_3 receives the packet, it identifies S_1 as its best relay node. It then forwards the packet to the sensor node succeeding it in the path included within the packet, notably S_4 . Before doing so, S_3 removes itself from the path, updating it to $[S_4]$. When S_4 receives the packet, it determines that S_3 is its best relay node.

The following section presents the performance evaluation of NBSDN, where its effectiveness is analyzed through comparison with established baseline routing algorithms under diverse network scales and topologies.

VI. PERFORMANCE EVALUATION

A. SIMULATION PARAMETERS AND METRICS

Our proposed routing algorithm, NBSDN, was evaluated against four established routing approaches:

- EASDN [6], an energy-aware SDN-based routing algorithm designed to optimize network lifetime by balancing energy consumption across sensor nodes.
- RLSDN [7], a reinforcement learning-based routing algorithm for the SDN controller.
- SPSDN [3], Dijkstra's shortest path-based routing algorithm for the SDN controller.
- NB-SDWSN [20], an earlier version of the Naïve Bayes-based routing algorithm that uses distance to the sink and residual energy for selecting relay nodes.

We implemented these routing approaches, including our routing algorithm, within the SDN-WISE framework and

assessed their performance through simulations conducted in the COOJA simulator [46] over a duration of 720 seconds. We evaluated each routing algorithm in static WSNs of varying sizes: 11, 21, 31, 41, 51, 61, 71, 81, 91, and 101 nodes. For each network size, the simulation was run 15 times, with each run using a different pseudo-random seed to generate a distinct network topology. This procedure ensures that the performance assessment reflects a broad range of configurations and does not depend on a particular topology instance. It also enabled the computation of 95% confidence intervals for the performance metrics, thereby ensuring the statistical validity of the reported results.

The main assumptions and simulation parameters used in our comparative study are listed below. These assumptions are consistent with those commonly adopted in the related literature [3], [6], [7], [20]. Moreover, the parameter values were maintained as originally defined in the SDN-WISE framework [3].

- The WSNs are composed of a single sink node and multiple sensor nodes, initially positioned randomly within a $100 \times 100 \text{ m}^2$ square area.
- The sink and all sensor nodes remain stationary at their initial deployment positions for the entire duration of the simulation.
- Every $T_{\text{beacon packet}} = 10$ seconds, each node generates a beacon packet and sends it to the nodes within its transmission range.
- Every $T_{\text{data packet}} = 10$ seconds, each sensor node generates and forwards a 14-byte data packet to its relay node. All the data packets are ultimately destined for the sink.
- Every $T_{\text{report packet}} = 20$ seconds, each sensor node generates and transmits a report packet to its relay node. The report packets are ultimately addressed to the controller through the sink. The sink, in contrast, creates and transmits its own report packet directly to the controller.
- Every $T_{\text{controller routing}} = 60$ seconds, the controller executes a specified routing algorithm (EASDN, RLSDN, SPSDN, NBSDN, or NB-SDWSN).
- The sensor and sink nodes employ wireless links based on the IEEE 802.15.4 standard [23] to transmit packets to their one-hop neighboring nodes. The MAC sublayer employs the unslotted CSMA/CA algorithm, which is defined by the IEEE 802.15.4 standard with its default settings [47]. Although the physical layer does not introduce transmission errors, packet collisions can occur when neighboring nodes transmit simultaneously, leading to packet loss.
- The Unit Disk Graph Medium (UDGM) propagation model [48] is used to model distance-dependent signal attenuation in the wireless communication medium. This model defines two distinct zones: the transmission range within which nodes can successfully receive transmitted messages and the interference range where nodes may experience signal interference. Nodes located beyond the interference range are not affected.

TABLE 6. Summary of the parameters and their corresponding values used in the simulations.

Parameters	Values
Simulation time (seconds)	720
Network size	11 to 101 nodes
Node deployment area	$100 \times 100 \text{ m}^2$
Node placement	Random
$T_{\text{beacon packet}}$ (seconds)	10
$T_{\text{data packet}}$ (seconds)	10
Data packet size (bytes)	14
$T_{\text{report packet}}$ (seconds)	20
$T_{\text{controller routing}}$ (seconds)	60
MAC protocol	IEEE 802.15.4
Wireless channel bandwidth (Kbps)	250
Radio medium	UDGM
Node transmission range (meters)	5
Node interference range (meters)	0
Node buffer capacity (packets)	100
Node initial battery level (joules)	5000

- The transmission range of the wireless medium is 5 meters, whereas the interference range is 0 meters (i.e., the interference range is not considered).
- The processing time of both the data and control packets is negligible.

Table 6 summarizes the key parameters and their associated values used in our simulations.

To compare the performance of the routing algorithms, we employ the following metrics:

- **Load balancing:** illustrates how a sensor node distributes its traffic, destined for the sink, among neighboring nodes when a specific routing algorithm is used by the controller. It is calculated by tracking the number of packets that each sensor node forwards to each of its neighbors.
- **Generated control traffic:** measures the average length of the control packets produced by the routing algorithm, namely those sent from the controller to the sink. This average is computed by dividing the total length of the packets by their overall count. Additionally, this metric quantifies the overall number and length of control packets generated within the WSN after a given routing algorithm is applied by the controller. It does so by separately summing the total number and length of control packets that each node has issued upon receiving packets initiated by the routing algorithm.
- **Network lifetime:** denotes the time interval between the activation of the WSN (i.e., when all the sensor and sink nodes are switched on) and the moment when the first sensor node depletes its battery and becomes non-functional. During this period, the WSN remains fully operational, with all sensor nodes active, ensuring complete sensing coverage of the monitored area and uninterrupted connectivity between all sensor nodes and the sink. The network remains fully functional until the first sensor node drains its battery and shuts down. This event marks the beginning of performance

TABLE 7. Traffic distribution among neighboring nodes under different routing algorithms (EASDN, RLSDN, SPSDN, NBSDN, AND NB-SDWSN). Values represent the amount of traffic each neighboring node received from the sensor nodes.

Sensor nodes	Total sent packets	Neighbors	EASDN	RLSDN	SPSDN	NBSDN	NB-SDWSN
$S_4 S_5 S_6$	192	S_2	71	121	0	93	98
		S_3	121	71	192	99	94
		S_4	0	57	0	63	54
$S_9 S_{10} S_{11} S_{12}$	256	S_5	0	45	0	46	53
		S_6	0	35	0	45	52
		S_7	124	59	256	46	48
		S_8	132	60	0	56	49
		S_9	0	6	0	27	21
$S_{14} S_{15}$	128	S_{10}	0	18	0	19	20
		S_{11}	0	34	0	27	21
		S_{12}	0	35	0	27	34
		S_{13}	128	35	128	28	32

degradation, as the loss of even a single node can compromise the sensing coverage and disconnect parts of the network from the sink, resulting in packet loss.

- **Packet loss:** measures the percentage of data packets transmitted by sensor nodes that fail to reach the sink. It is calculated by dividing the difference between the total number of data packets sent by the sensor nodes and the number of data packets successfully delivered to the sink by the total number of data packets sent.
- **Data throughput:** indicates the rate at which data packets were successfully delivered from the sensor nodes to the sink during the simulation time. It is computed by dividing the number of data packets that successfully reached the sink by the simulation time.
- **Network latency:** represents the average time taken by a data packet generated by a sensor node to reach the sink. It is calculated by summing the time taken by each generated data packet to arrive at the sink and dividing this total by the number of generated packets.

B. RESULTS AND DISCUSSIONS

1) LOAD BALANCING

Table 7 shows the distribution of sensor node traffic (i.e., data packets) among neighboring nodes across four simulation runs, with the controller operating EASDN, RLSDN, SPSDN, NBSDN, and NB-SDWSN, respectively.

Each simulation was conducted on the same WSN, consisting of 15 nodes arranged as shown in Fig. 5. In this WSN, the sink S_1 has the blue-colored sensor nodes as neighbors. The yellow-colored sensor nodes are connected to the sensor nodes in the levels immediately above and below, excluding blue-colored nodes in the lower level. The blue-colored nodes are connected to the sink and to the sensor nodes in the level directly below, excluding those that are also blue-colored. Note that in Fig. 5, the value shown close to each node

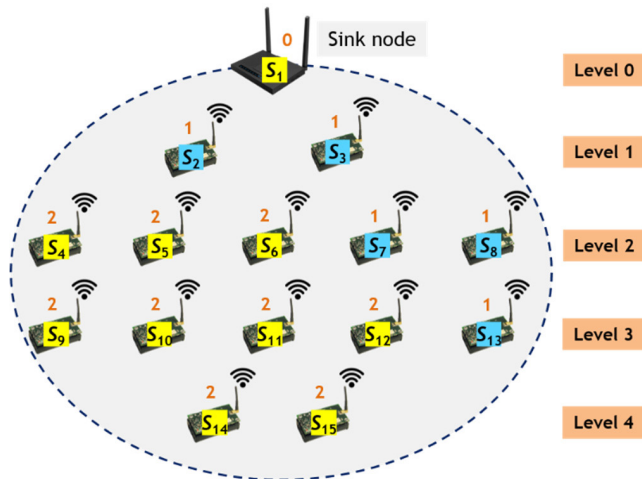


FIGURE 5. A WSN topology organized by levels, containing 15 nodes with their respective distances (in hops) to the sink.

represents its distance to the sink, expressed as the number of hops a packet transmitted by that node must traverse to reach the sink.

Given the illustrated WSN topology, the controller can make the following routing decisions:

- For each of the nodes S_4 , S_5 , and S_6 , it may assign either S_2 or S_3 as the relay node.
- For each of the nodes S_9 , S_{10} , S_{11} , and S_{12} , it may designate either the neighbor S_4 , S_5 , S_6 , S_7 , or S_8 as the relay node.
- For each of the nodes S_{14} and S_{15} , it may select either the neighbor S_9 , S_{10} , S_{11} , S_{12} , or S_{13} as the relay node.
- For the blue-colored sensor nodes, since they are directly connected to the sink S_1 , the controller always assigns S_1 as their relay node.

From Table 7, we observe that when the controller employs SPSPDN, the sensor nodes fail to distribute their data packets across multiple neighboring nodes and instead transmit all packets through a single neighbor. The lack of a load-balancing mechanism in SPSPDN stems from its strategy of repeatedly assigning, for each sensor node in the WSN, the same neighbor as its relay node to route its packets toward the sink. This behavior is clearly illustrated in Table 7, where SPSPDN consistently selects S_{13} as the relay node for both nodes S_{14} and S_{15} , despite the presence of other neighbors (S_9 , S_{10} , S_{11} , and S_{12}). This is demonstrated by the fact that S_9 , S_{10} , S_{11} , and S_{12} do not receive any data packets from S_{14} or S_{15} , whereas S_{13} alone receives all 128 data packets transmitted by these two nodes.

We also observe from Table 7 that, in contrast to SPSPDN, EASDN allows sensor nodes to balance their data packets among multiple neighboring nodes. However, this balancing is only partial (which will be referred to as partial load balancing), being limited to neighbors located close to the sink. This is because, for each sensor node, EASDN periodically selects a different neighboring node to serve as its relay, but the

selection is restricted to those closest to the sink. For instance, for nodes S_9 , S_{10} , S_{11} , and S_{12} , EASDN consistently selects either S_7 or S_8 as their relay node, alternating between the two despite the availability of other neighbors, specifically S_4 , S_5 , and S_6 . This preference is due to S_7 and S_8 being only one hop away from the sink, whereas S_4 , S_5 , and S_6 are two hops away. Supporting this observation, S_4 , S_5 , and S_6 receive no data packets from S_9 to S_{12} , whereas S_7 and S_8 exclusively receive their packets, with nearly equal counts of 124 and 132, respectively.

Another observation from Table 7 is that when a node has only one neighboring node closest to the sink, EASDN behaves identically to SPSPDN, repeatedly selecting that neighbor as the relay. This results in unbalanced data transmission as all packets are sent through that single neighbor. For example, S_{14} and S_{15} have neighbors S_9 to S_{13} . Among these, only S_{13} is one hop away from the sink, while the others are two hops away. It is evident that EASDN, like SPSPDN, consistently selects S_{13} as the relay node for S_{14} and S_{15} , as it receives all 128 data packets transmitted by both nodes.

In contrast to EASDN, the other routing algorithms—NBSPDN, NB-SDWSN, and RLSPDN—allow sensor nodes to distribute their data packets across all neighboring nodes, achieving global load balancing, as clearly illustrated in Table 7. For example, nodes S_{14} and S_{15} distribute their traffic almost evenly among all their neighboring nodes (S_9 , S_{10} , S_{11} , S_{12} , and S_{13}) when the controller employs NBSPDN, NB-SDWSN, or RLSPDN. This is supported by the fact that these nodes receive data packets from S_{14} and S_{15} in approximately equal numbers: 27, 19, 27, 27, and 28 in NBSPDN, 21, 20, 21, 34, and 32 in NB-SDWSN, and 6, 18, 34, 35, and 35 in RLSPDN, respectively. The reason NBSPDN, NB-SDWSN, and RLSPDN allow sensor nodes to distribute their traffic almost evenly across all neighboring nodes is that, for each sensor node in the WSN, they periodically rotate the role of relay among all its neighbors, not just those closest to the sink, as in EASDN.

2) GENERATED CONTROL TRAFFIC

Fig. 6 illustrates the average length (in bytes) of control packets generated by the five competing routing algorithms—EASDN, RLSPDN, SPSPDN, NB-SDWSN, and NBSPDN—as the network size increases from 11 to 101 nodes in increments of 10. NB-SDWSN and NBSPDN consistently produce the shortest control packets across all network sizes, with average lengths ranging from 16 to 61 bytes. In contrast, RLSPDN generates the longest control packets, exceeding 100 bytes for network sizes of 51 to 101 nodes. EASDN follows next, producing packets slightly shorter than those of RLSPDN but still significantly longer than those of SPSPDN, NB-SDWSN, and NBSPDN. This disparity in the lengths of control packets arises from the content of the control packets: RLSPDN and EASDN embed a complete routing table in each packet, mapping every sensor node to its assigned relay node. In comparison, SPSPDN, NB-SDWSN, and NBSPDN include only

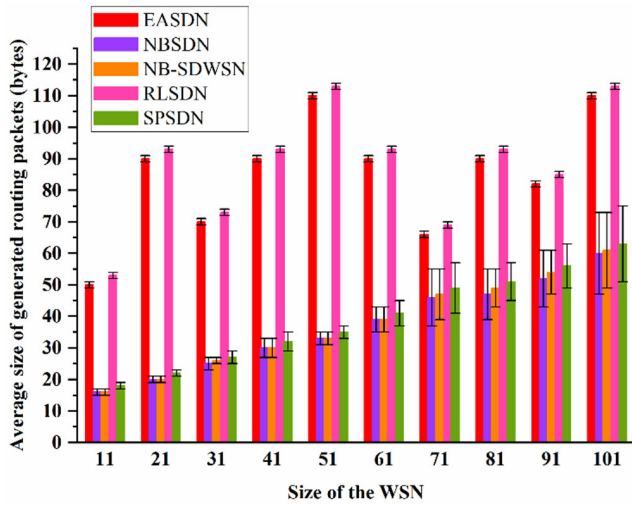


FIGURE 6. Length of control packets generated by the controller in EASDN, NBSDN, NB-SDWSN, RLSDN, and SPSPDN.

a single routing path per packet, resulting in substantially shorter packet lengths.

Nonetheless, NB-SDWSN and NBSDN generate slightly shorter routing packets than SPSPDN. This is because, before encapsulating a routing path (e.g., $S_1, S_2, \dots, S_i$) into a packet and transmitting it to the sink S_1 , both NB-SDWSN and NBSDN exclude S_1 from the path. The next node in the path (i.e., S_2) can implicitly infer that S_1 is its relay node, as it directly receives the routing packet from S_1 . Consequently, it is unnecessary to explicitly indicate to S_2 its relay node by including it in the routing path, which reduces the length of the control packets. In contrast, SPSPDN does not implement this optimization and retains the entire routing path within the packet.

Figs. 7 and 8 present the total number and length (in bytes), respectively, of control packets generated by the nodes within the WSNs when using EASDN, RLSDN, SPSPDN, NB-SDWSN, and NBSDN at the controller. RLSDN produces the highest control traffic overhead due to its reliance on RL, which requires frequent control packet exchanges between the controller and the nodes to determine the optimal routing tables. An additional contributing factor is that RLSDN generates broadcast-based routing control messages, resulting in redundant transmissions and a substantial increase in overhead. EASDN generates less control overhead than RLSDN but still exhibits a notable increase as the network size scales. Similar to RLSDN, its routing control packets are broadcast-based, which significantly contributes to the overall control overhead.

Contrastingly, SPSPDN, NB-SDWSN, and NBSDN substantially reduce control overhead, as they utilize unicast control messages, thereby avoiding redundant retransmissions, and thus minimizing total control traffic. Nevertheless, NB-SDWSN and NBSDN achieve lower control traffic overhead than SPSPDN. This is because SPSPDN generates and transmits, for each sensor node in the WSN, a routing control

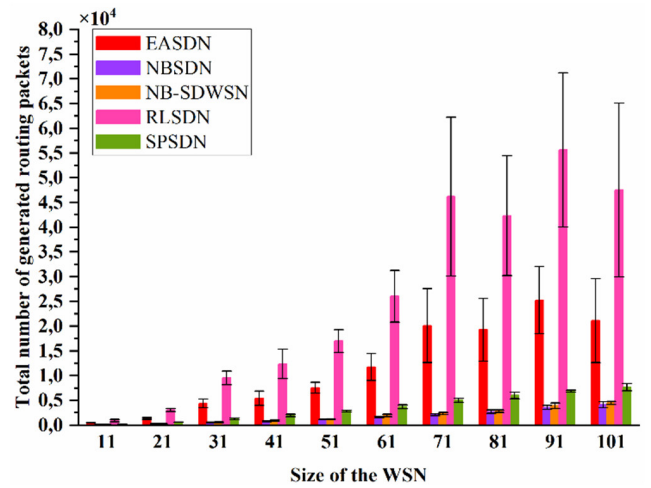


FIGURE 7. Total number of control packets generated within the WSN in EASDN, NBSDN, NB-SDWSN, RLSDN, and SPSPDN.

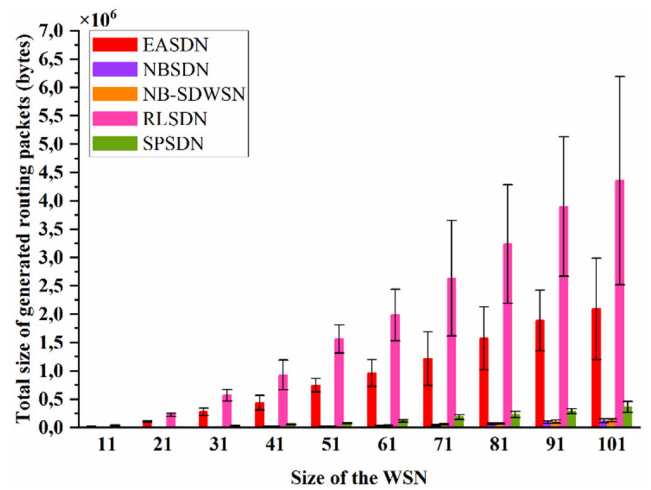


FIGURE 8. Length of control packets generated within the WSN in EASDN, NBSDN, NB-SDWSN, RLSDN, and SPSPDN.

packet containing a routing path from which the node determines its relay node. Conversely, NB-SDWSN and NBSDN, by leveraging Algorithm 2, generate and transmit a minimal number of routing control packets with the objective that a single control packet enables multiple nodes in the WSN to identify their respective relay nodes without requiring separate transmissions to each node.

3) NETWORK LIFETIME

Fig. 9 depicts the WSN lifetime, expressed in milliseconds, as a function of network size for the five routing algorithms. NBSDN consistently attains the longest lifetime across all network scales, followed by NB-SDWSN, EASDN, RLSDN, and SPSPDN. The superior performance behavior of NBSDN and NB-SDWSN over the other algorithms is attributed, first, to their ability to balance the load evenly among neighboring sensor nodes, resulting in uniform energy consumption and preventing premature energy depletion in individual nodes.

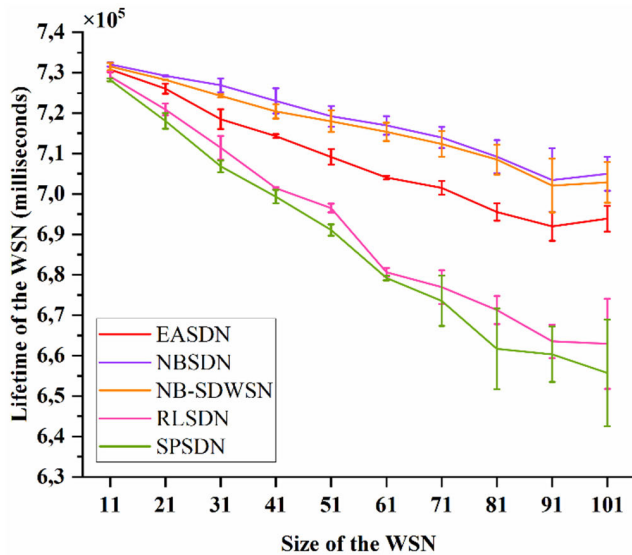


FIGURE 9. Network lifetime in EASDN, NBSDN, NB-SDWSN, RLSDN, and SPSDN.

This leads to increased network longevity and extended operational time. Second, both NBSDN and NB-SDWSN generate minimal control traffic overhead within the network, thereby preserving node energy and further contributing to an extended network lifetime. However, the lifetime of the WSN is further prolonged when the controller employs NBSDN compared to NB-SDWSN. This improvement is attributed to the fact that NBSDN, unlike NB-SDWSN, considers the link quality parameter when selecting a relay node for each sensor node. Consequently, NBSDN, in contrast to NB-SDWSN, avoids the selection of relay nodes with poor link quality, enabling sensor nodes to successfully transmit packets to their relay nodes on the first attempt without transmission errors. This eliminates the need for retransmissions, thereby conserving node energy and ultimately extending the network lifetime. On the other hand, EASDN achieves a shorter network lifetime compared to NBSDN and NB-SDWSN, as it performs only partial load balancing and introduces significant control overhead. RLSDN, despite promoting global load balancing, performs worse than EASDN due to its substantially higher control traffic overhead. SPSDN exhibits the shortest network lifetime due to the absence of any load-balancing mechanism. This limitation causes rapid energy depletion in specific nodes, particularly those in proximity to the sink, resulting in premature network shutdown.

4) PACKET LOSS

Fig. 10 presents the packet loss rates across various network sizes for the controller-based routing algorithms: EASDN, RLSDN, SPSDN, NB-SDWSN, and NBSDN. The results indicate that NBSDN achieves the lowest packet loss, maintaining stable delivery ratios as the network size increases, followed by NB-SDWSN. RLSDN and EASDN exhibit higher packet loss rates than NBSDN and NB-SDWSN,

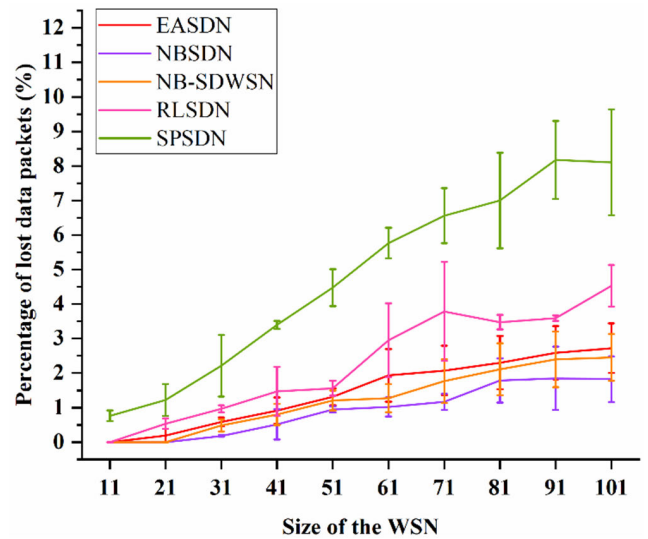


FIGURE 10. Packet loss rate in EASDN, NBSDN, NB-SDWSN, RLSDN, and SPSDN.

primarily due to the generation of longer routing control packets, as shown in Fig. 6. Such control packets lead to early energy depletion of nodes located immediately near the sink, as they are the first to receive them. The subsequent depletion of these nodes leads to the isolation of other nodes from the sink, preventing them from delivering their packets and thereby increasing packet loss. SPSDN experiences the highest packet loss, especially in dense networks. This behavior is attributed to the absence of a load-balancing mechanism, which causes uneven energy consumption and the premature depletion of specific nodes. Consequently, these nodes become nonfunctional earlier than the others, and the nodes relying on them as relays remain isolated for extended periods. During this time, packets transmitted by the isolated nodes fail to reach the sink, contributing significantly to packet loss.

In contrast, NBSDN demonstrates superior robustness to packet loss and sustains higher data delivery reliability, even in large-scale deployments. This performance advantage is due to its integrated load-balancing mechanism, which promotes uniform energy consumption among sensor nodes. As a result, nodes tend to deplete their energy and become non-functional nearly simultaneously. When the relay nodes fail, the dependent sensor nodes remain active but are isolated only briefly, during which they continue transmitting packets. This short isolation period leads to a limited number of lost packets. Although NB-SDWSN enables global load balancing similar to NBSDN, as demonstrated in Table 7, it exhibits a higher packet loss rate. This difference in performance is due to the fact that NBSDN, unlike NB-SDWSN, incorporates buffer occupancy as a feature in the relay node selection procedure. Consequently, NBSDN avoids choosing relay nodes with loaded buffers, thereby mitigating node overload and congestion and thus reducing packet loss.

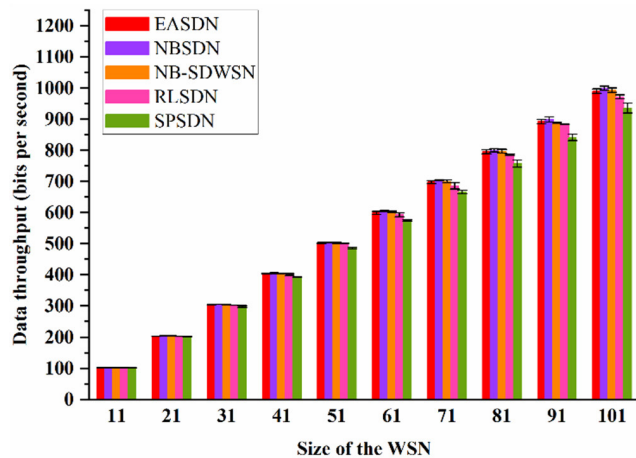


FIGURE 11. Data throughput in EASDN, NBSDN, NB-SDWSN, RLSDN, and SPSDN.

5) DATA THROUGHPUT

Fig. 11 shows the data throughput, measured in bits per second, obtained in WSNs of different sizes across controller configurations based on EASDN, RLSDN, SPSDN, NB-SDWSN, and NBSDN. The results reveal that the throughput increases for the five routing algorithms as the network size increases. NBSDN consistently achieves the highest throughput, particularly in dense network scenarios, due to its ability to extend network lifetime and reduce packet loss, thereby enabling the transmission and successful delivery of a greater number of data packets from sensor nodes to the sink. NB-SDWSN exhibits slightly lower throughput than NBSDN but outperforms EASDN, RLSDN, and SPSDN, owing to its improved network longevity and reduced packet loss rate. EASDN is ranked next to NB-SDWSN, surpassing RLSDN and SPSDN, as it prolongs the network operational time and mitigates packet loss more effectively than the other approaches. Nonetheless, SPSDN has the lowest throughput, primarily due to its shortest network operation time and the highest packet loss rate among the evaluated routing algorithms.

6) NETWORK LATENCY

Fig. 12 reports the end-to-end latency, in milliseconds, for WSNs of different sizes when the controller operates with EASDN, RLSDN, SPSDN, NB-SDWSN, and NBSDN. Among the five competing algorithms, NBSDN achieves the lowest network latency across all network sizes. This improvement stems from NBSDN's joint consideration of node buffer occupancy and link quality parameters when selecting relay nodes, unlike NB-SDWSN and the other algorithms. By accounting for buffer occupancy, NBSDN prevents the selection of relay nodes with heavily loaded buffers, ensuring that traffic is forwarded through nodes with lighter buffer utilization. This reduces packet queuing delays at the relay nodes and, consequently, the overall network latency. Furthermore, by considering link quality, NBSDN

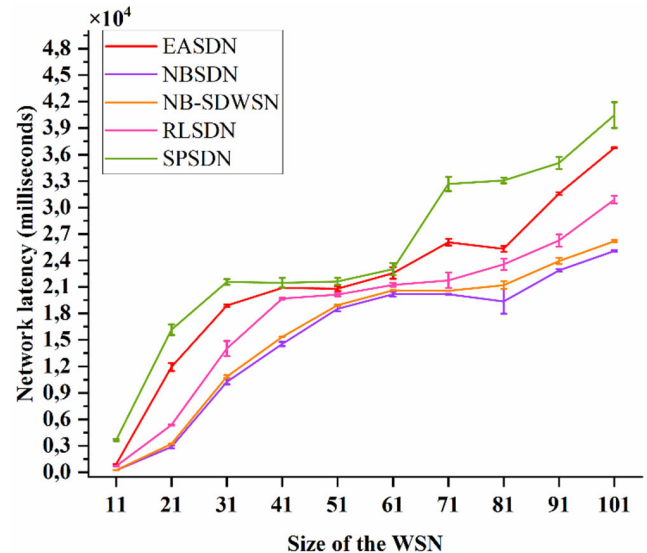


FIGURE 12. Network latency in EASDN, NBSDN, NB-SDWSN, RLSDN, and SPSDN.

avoids designating relay nodes with poor communication links, thereby minimizing packet retransmissions due to collisions and associated delays, which further contribute to latency reduction.

NB-SDWSN achieves the second-lowest network latency, outperforming RLSDN, EASDN, and SPSDN. Its performance advantage is attributed to its effective global load-balancing mechanism, which prevents nodes from persistently forwarding traffic through the same neighbor or a limited subset of neighbors. This mitigates traffic concentration, reduces excessive buffer occupancy at individual relay nodes, and shortens queuing delays. RLSDN exhibits lower latency than EASDN and SPSDN but higher than NB-SDWSN. Although RLSDN supports global load balancing, its substantial control traffic overhead increases buffer utilization at nodes. Consequently, incoming packets experience additional waiting time due to buffer space occupied by control messages. EASDN, despite generating less control traffic than RLSDN, shows higher latency because it employs only a partial load-balancing mechanism. As a result, network traffic tends to converge on the same subset of neighbors, causing rapid buffer buildup and longer queuing delays at intermediate nodes. Finally, SPSDN has the highest network latency among the routing algorithms, primarily due to the lack of a load-balancing mechanism. All packets generated by a node are forwarded through a single neighbor, causing rapid buffer congestion at that node. This congestion increases queuing time for newly arriving packets, thereby elevating the overall network latency.

Let us conclude this section by reflecting on the overall findings and implications of the above comparative evaluation. The results demonstrate that the proposed NBSDN algorithm offers notable performance gains under the examined static WSN scenarios. Specifically, NBSDN consistently

achieves effective global load balancing by evenly distributing traffic among neighboring nodes, thereby reducing congestion and extending network lifetime. It also introduces minimal control overhead, generating the fewest and shortest control packets among all evaluated algorithms, and maintaining stable overhead as the network size scales up. Although NBSDN relies on periodic routing updates (every 60 seconds), which can limit responsiveness to sudden topological changes such as node failures or traffic bursts, it nevertheless demonstrates a favorable trade-off between routing efficiency, scalability, and control overhead in static SDN-based WSNs.

It is important to note that this study has focused exclusively on static topologies, in line with prior research work on SPSDN [3], EASDN [6], RLSDN [7], NB-SDWSN [20], and other related routing protocols [4], [9], [12], [30], [32], due to constraints in computational resources and the excessive simulation time required to accurately model mobility. While NBSDN is expected to retain its relative performance advantage under low-mobility conditions, its effectiveness—like that of the other competing algorithms—is likely to degrade as node mobility increases, primarily due to outdated routing information and elevated control overhead. In such dynamic settings, adaptive or predictive routing strategies may be necessary to sustain performance. Nonetheless, the findings presented in this study provide strong evidence of NBSDN's suitability for practical deployments where mobility is limited or infrequent. Such scenarios are typical of many real-world WSN applications, including environmental monitoring, industrial automation, structural health monitoring, pipeline surveillance, and precision agriculture, where sensor nodes are generally stationary or exhibit negligible movement over time [49].

VII. CONCLUSION AND FUTURE WORK

This study has introduced an enhanced Naïve Bayes-based routing algorithm (referred to as NBSDN) for the SDN controller, designed to overcome the limitations of existing routing protocols developed for Software-Defined Wireless Sensor Networks (SDWSNs). NBSDN leverages probabilistic decision-making to optimize routing by selecting relay nodes based on key features, including residual energy, distance to the sink, buffer occupancy, and link quality. By utilizing the centralized decision-making capabilities of the SDN controller, NBSDN effectively balances traffic loads among neighboring nodes and reduces control traffic overhead. These mechanisms collectively contribute to an extended network lifespan and improved network performance.

This work presented one of the first comprehensive evaluations of routing algorithms in SDWSNs, addressing a notable gap in the existing literature. While prior research has largely focused on small-scale networks and a limited set of performance metrics, this study assessed network performance across six key metrics: load balancing, con-

trol traffic generation, network lifetime, packet loss, data throughput, and network latency. Notably, load balancing and control traffic—metrics often omitted in existing SDWSN routing studies—are included here due to their critical impact on overall network performance, providing a more complete and robust evaluation. Simulations across both small and large network scenarios have demonstrated that NBSDN consistently outperforms established algorithms, including Dijkstra's shortest path (SPSDN), Energy-Aware SDN (EASDN), Naïve Bayes for SDWSN (NB-SDWSN), and Reinforcement Learning-based routing for the SDN controller (RLSDN). The results confirm NBSDN's effectiveness, achieving longer network lifetime, reduced packet loss, lower control overhead, decreased latency, and higher data throughput across diverse network configurations. The findings of this study highlight the potential of NBSDN as an efficient and reliable approach to routing in SDWSNs. The ability of NBSDN to adapt to dynamic network conditions and optimize resource utilization offers a scalable and practical solution for IoT applications, particularly in energy-constrained environments.

Future work will explore several directions for further enhancement and validation. These include the evaluation of NBSDN in mobile network scenarios and the integration of additional features, such as mobility prediction and link stability, to enable the algorithm to operate effectively in mobile network environments. Real-world experimental validations will also be performed to assess the practical applicability and robustness of NBSDN in large-scale and mobile IoT and SDWSN deployments.

REFERENCES

- [1] S. R. J. Ramson, S. Vishnu, and M. Shanmugam, "Applications of Internet of Things (IoT)—An overview," in *Proc. 5th Int. Conf. Devices, Circuits Syst. (ICDCS)*, Coimbatore, India, Mar. 2020, pp. 92–95.
- [2] F. Bannour, S. Souihi, and A. Mellouk, *Software-Defined Networking 2: Extending SDN Control To Large-Scale Networks*. Hoboken, NJ, USA: Wiley, 2022, doi: [10.1002/9781394186181](https://doi.org/10.1002/9781394186181).
- [3] L. Galluccio, S. Milardo, G. Morabito, and S. Palazzo, "SDN-WISE: Design, prototyping and experimentation of a stateful SDN solution for wireless sensor networks," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2015, pp. 513–521.
- [4] M. Baddeley, R. Nejabati, G. Oikonomou, M. Sooriyabandara, and D. Simeonidou, "Evolving SDN for low-power IoT networks," in *Proc. 4th IEEE Conf. Netw. Softwarization Workshops (NetSoft)*, Jun. 2018, pp. 71–79.
- [5] J. Wang, Y. Miao, P. Zhou, M. S. Hossain, and S. M. M. Rahman, "A software defined network routing in wireless multihop network," *J. Netw. Comput. Appl.*, vol. 85, pp. 76–83, May 2017, doi: [10.1016/j.jnca.2016.12.007](https://doi.org/10.1016/j.jnca.2016.12.007).
- [6] M. Younus, S. Islam, and S. Kim, "Proposition and real-time implementation of an energy-aware routing protocol for a software defined wireless sensor network," *Sensors*, vol. 19, no. 12, p. 2739, Jun. 2019, doi: [10.3390/s19122739](https://doi.org/10.3390/s19122739).
- [7] M. U. Younus, M. K. Khan, and A. R. Bhatti, "Improving the software-defined wireless sensor networks routing performance using reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3495–3508, Mar. 2022, doi: [10.1109/JIOT.2021.3102130](https://doi.org/10.1109/JIOT.2021.3102130).
- [8] M. U. Younus, M. K. Khan, M. R. Anjum, S. Afridi, Z. A. Arain, and A. A. Jamali, "Optimizing the lifetime of software defined wireless sensor network via reinforcement learning," *IEEE Access*, vol. 9, pp. 259–272, 2021, doi: [10.1109/ACCESS.2020.3046693](https://doi.org/10.1109/ACCESS.2020.3046693).

- [9] D. Godfrey, B. Suh, B. H. Lim, K.-C. Lee, and K.-I. Kim, "An energy-efficient routing protocol with reinforcement learning in software-defined wireless sensor networks," *Sensors*, vol. 23, no. 20, p. 8435, Oct. 2023, doi: [10.3390/s23208435](https://doi.org/10.3390/s23208435).
- [10] A. Narwaria, V. Kumari, and A. P. Mazumdar, "RL-EAR: Reinforcement learning-based energy-aware routing for software-defined wireless sensor network," *J. Supercomput.*, vol. 81, no. 3, p. 485, Feb. 2025, doi: [10.1007/s11227-025-06998-1](https://doi.org/10.1007/s11227-025-06998-1).
- [11] A. K. Shakya, G. Pillai, and S. Chakrabarty, "Reinforcement learning algorithms: A brief survey," *Expert Syst. Appl.*, vol. 231, Nov. 2023, Art. no. 120495, doi: [10.1016/j.eswa.2023.120495](https://doi.org/10.1016/j.eswa.2023.120495).
- [12] V. Tyagi and S. Singh, "GM-WOA: A hybrid energy efficient cluster routing technique for SDN-enabled WSNs," *J. Supercomput.*, vol. 79, no. 13, pp. 14894–14922, Apr. 2023, doi: [10.1007/s11227-023-05263-7](https://doi.org/10.1007/s11227-023-05263-7).
- [13] M. Masood, M. M. Fouad, S. Seyedzadeh, and I. Glesk, "Energy efficient software defined networking algorithm for wireless sensor networks," in *Proc. 13th Int. Scientific Conf. Sustain., Modern Safe Transp.*, May 2019, pp. 1481–1488.
- [14] J. Shreyas, D. Chouhan, M. Harshitha, P. K. Udayaprasad, and S. M. D. Kumar, "Network lifetime enhancement routing algorithm for IoT enabled software defined wireless sensor network," in *Proc. 3rd Int. Conf. Sustain. Adv. Comput.*, Bangalore, India, Mar. 2022, pp. 499–508.
- [15] R. Samadi, A. Nazari, and J. Seitz, "Intelligent energy-aware routing protocol in mobile IoT networks based on SDN," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 4, pp. 2093–2103, Dec. 2023, doi: [10.1109/TGCN.2023.3296272](https://doi.org/10.1109/TGCN.2023.3296272).
- [16] R. J. Kavitha and P. Anandavalli, "An efficient scyphozoa swarm optimization and fuzzy density-based clustering routing for underwater wireless sensor networks," *Tehnički Vjesnik*, vol. 31, no. 5, pp. 1589–1595, Oct. 2024, doi: [10.17559/TV-20231121001131](https://doi.org/10.17559/TV-20231121001131).
- [17] N. Kumar and D. P. Vidyarthi, "A green routing algorithm for IoT-enabled software defined wireless sensor network," *IEEE Sensors J.*, vol. 18, no. 22, pp. 9449–9460, Nov. 2018, doi: [10.1109/JSEN.2018.2869629](https://doi.org/10.1109/JSEN.2018.2869629).
- [18] R. Ramteke, S. Singh, and A. Malik, "Optimized routing technique for IoT enabled software-defined heterogeneous WSNs using genetic mutation based PSO," *Comput. Standards Interfaces*, vol. 79, Jan. 2022, Art. no. 103548, doi: [10.1016/j.csi.2021.103548](https://doi.org/10.1016/j.csi.2021.103548).
- [19] M. A. Tawfeek, I. Alrashdi, M. Alruwaili, L. Jamel, G. F. Elhady, and H. Elwahsh, "Improving energy efficiency and routing reliability in wireless sensor networks using modified ant colony optimization," *EURASIP J. Wireless Commun. Netw.*, vol. 2025, no. 1, p. 22, Apr. 2025, doi: [10.1186/s13638-025-02449-w](https://doi.org/10.1186/s13638-025-02449-w).
- [20] A. Tcherak, S. Loucif, and M. O. Khaoua, "On efficient routing for SDN-based wireless sensor networks," in *Proc. 24th Int. Arab Conf. Inf. Technol. (ACIT)*, Ajman, United Arab Emirates, Dec. 2023, pp. 1–6.
- [21] I. E. Naqa and M. J. Murphy, "What is machine learning?" in *Machine Learning in Radiation Oncology: Theory and Applications*. Cham, Switzerland: Springer, Jan. 2015, pp. 3–11.
- [22] P. J. B. Pajila, B. G. Sheena, A. Gayathri, J. Aswini, M. Nalini, and R. S. Subramanian, "A comprehensive survey on naive Bayes algorithm: Advantages, limitations and applications," in *Proc. 4th Int. Conf. Smart Electron. Commun. (ICOSEC)*, Trichy, India, Sep. 2023, pp. 1228–1234.
- [23] *Wireless Medium Access Control (MAC) and Physical Layer (PHY) Specifications for Low-Rate Wireless Personal Area Networks (LR-WPANs)*, Standard Standard 2006:802.4, 2006.
- [24] N. McKeown, "OpenFlow," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Mar. 2008, doi: [10.1145/1355734.1355746](https://doi.org/10.1145/1355734.1355746).
- [25] S. Chakrabarty, D. W. Engels, and S. Thathapudi, "Black SDN for the Internet of Things," in *Proc. IEEE 12th Int. Conf. Mobile Ad Hoc Sensor Syst.*, Oct. 2015, pp. 190–198.
- [26] A. El Yazidi, M. El Kamili, and M. L. Hasnaoui, "Black SDN for WSN," in *Proc. 1st Int. Conf. Smart Syst. Data Sci. (ICSSD)*, Rabat, Morocco, Oct. 2019, pp. 1–4.
- [27] M. U. Farooq, X. Wang, A. Hawbani, L. Zhao, A. Al-Dubai, and O. Busaileh, "SDORP: SDN based opportunistic routing for asynchronous wireless sensor networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 8, pp. 4912–4929, Aug. 2023, doi: [10.1109/TMC.2022.3158695](https://doi.org/10.1109/TMC.2022.3158695).
- [28] F. S. Gharehchopogh and H. Gholizadeh, "A comprehensive survey: Whale optimization algorithm and its applications," *Swarm Evol. Comput.*, vol. 48, pp. 1–24, Aug. 2019.
- [29] T. Dokeroglu, E. Sevinc, T. Kucukyilmaz, and A. Cosar, "A survey on new generation Metaheuristic algorithms," *Comput. Ind. Eng.*, vol. 137, Nov. 2019, Art. no. 106040, doi: [10.1016/j.cie.2019.106040](https://doi.org/10.1016/j.cie.2019.106040).
- [30] C.-K. Ke, M.-Y. Wu, W.-H. Hsu, and C.-Y. Chen, "Discover the optimal IoT packets routing path of software-defined network via artificial bee colony algorithm," in *Proc. 12th EAI Int. Conf. Wireless Internets*, Nov. 2019, pp. 147–162.
- [31] D. Karaboga and B. Basturk, "A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm," *J. Global Optim.*, vol. 39, no. 3, pp. 459–471, Apr. 2007, doi: [10.1007/s10898-007-9149-x](https://doi.org/10.1007/s10898-007-9149-x).
- [32] N. Abdolmaleki, M. Ahmadi, H. T. Malazi, and S. Milardo, "Fuzzy topology discovery protocol for SDN-based wireless sensor networks," *Simul. Model. Pract. Theory*, vol. 79, pp. 54–68, Dec. 2017, doi: [10.1016/j.simpat.2017.09.004](https://doi.org/10.1016/j.simpat.2017.09.004).
- [33] J. Clifton and E. Laber, "Q-learning: Theory and applications," *Annu. Rev. Statist. Appl.*, vol. 7, no. 1, pp. 279–301, Mar. 2020, doi: [10.1146/annurev-statistics-031219-041220](https://doi.org/10.1146/annurev-statistics-031219-041220).
- [34] Z. Abboud, M. Shuker, Ç. Aydin, and D. Ç. Atilla, "Extending wireless sensor Networks' lifetimes using deep reinforcement learning in a software-defined network architecture," *Academic Platform J. Eng. Sci.*, vol. 9, no. 1, pp. 39–46, Jan. 2021, doi: [10.21541/apjes.687496](https://doi.org/10.21541/apjes.687496).
- [35] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017, doi: [10.1109/MSP.2017.2743240](https://doi.org/10.1109/MSP.2017.2743240).
- [36] A. Prakash and S. Chauhan, "A comprehensive survey of trending tools and techniques in deep learning," in *Proc. Int. Conf. Disruptive Technol. (ICDT)*, May 2023, pp. 289–292.
- [37] Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A survey of convolutional neural networks: Analysis, applications, and prospects," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 6999–7019, Dec. 2022, doi: [10.1109/TNNLS.2021.3084827](https://doi.org/10.1109/TNNLS.2021.3084827).
- [38] T.-V.-T. Duong and L. H. Binh, "IRSML: An intelligent routing algorithm based on machine learning in software defined wireless networking," *ETRI J.*, vol. 44, no. 5, pp. 733–745, Aug. 2022, doi: [10.4218/etrij.2021-0212](https://doi.org/10.4218/etrij.2021-0212).
- [39] A. Bahmer, D. Gupta, and F. Effenberger, "Modern artificial neural networks: Is evolution cleverer?" *Neural Comput.*, vol. 35, no. 5, pp. 763–806, Apr. 2023, doi: [10.1162/neco_a_01575](https://doi.org/10.1162/neco_a_01575).
- [40] S. Chua, A. Tan, P. N. E. Nohuddin, and M. H. Ahmad Hijazi, "Comparing the effectiveness and efficiency of machine learning models for spam detection on Twitter," *J. Adv. Res. Appl. Sci. Eng. Technol.*, vol. 60, pp. 127–138, Oct. 2024, doi: [10.37934/araset.61.2.127138](https://doi.org/10.37934/araset.61.2.127138).
- [41] U. Hassan, Z. U. Rehman, I. Ahmad, and S. U. Islam, "Analyzing the energy consumption of random forest and support vector machine models: Paving the way for green and sustainable artificial intelligence," *Discover Internet Things*, vol. 5, no. 1, p. 100, Sep. 2025, doi: [10.1007/s43926-025-00188-4](https://doi.org/10.1007/s43926-025-00188-4).
- [42] D. Kalibiatienė and J. Miliauskaitė, "A hybrid systematic review approach on complexity issues in data-driven fuzzy inference systems development," *Informatica*, vol. 32, pp. 85–118, Jan. 2021, doi: [10.15388/21-infor444](https://doi.org/10.15388/21-infor444).
- [43] I. Wickramasinghe and H. Kalutara, "Naive bayes: Applications, variations and vulnerabilities: A review of literature with code snippets for implementation," *Soft Comput.*, vol. 25, no. 3, pp. 2277–2293, Sep. 2020, doi: [10.1007/s00500-020-05297-6](https://doi.org/10.1007/s00500-020-05297-6).
- [44] D. Berrar, *Bayes' Theorem and Naive Bayes Classifier in Encyclopedia of Bioinformatics and Computational Biology*. Amsterdam, The Netherlands: Elsevier, 2024, pp. 483–494.
- [45] Y. Chapre, P. Mohapatra, S. Jha, and A. Seneviratne, "Received signal strength indicator and its analysis in a typical WLAN system," in *Proc. 38th Annu. IEEE Conf. Local Comput. Netw.*, Oct. 2013, pp. 304–307.
- [46] C. Thomson, I. Romdhani, A. Y. Al Dubai, M. Qasem, B. Ghaleb, and I. Wadhaj, "Cooja Simulator Manual," Edinburgh Napier University, Dubai, U.K., Jul. 2016.
- [47] *IEEE Standard for Low-rate Wireless Networks*, Standard IEEE 802.15.4–2015, 2015.
- [48] F. Osterlind, A. Dunkels, J. Eriksson, N. Finne, and T. Voigt, "Cross-level sensor network simulation with COOJA," in *Proc. 31st IEEE Conf. Local Comput. Netw.*, Tampa, FL, USA, Nov. 2006, pp. 641–648.
- [49] S. El Khediri, "Wireless sensor networks: A survey, categorization, main issues, and future orientations for clustering protocols," *Computing*, vol. 104, no. 8, pp. 1775–1837, Mar. 2022.



and mobile applications, as well as the Internet of Things and machine learning-based applications, for a company in Algeria.

AMINE TCHERAK received the bachelor's degree in computer systems from the University of Algiers 1, Algeria, and the master's degree in networks and computer systems from the University of Blida 1, Algeria, where he is currently pursuing the Ph.D. degree in networks and distributed systems with the Department of Computer Science. He has taught the operating systems module with the Department of Computer Science, University of Blida 1. In addition, he was a Developer of web



Zayed University, she was an Associate Professor and the Chair with the Software Engineering Department, ALHOSN University; a Postdoctoral Research Fellow with the University of Moncton, New Brunswick, Canada; and an Assistant Professor with UAE University. She has actively contributed to the academic community as a member of organizing committees for several international conferences. She served on the program committees of over 30 international conferences and respected journals. Her current research interests include artificial intelligence (AI), machine learning, and deep learning, particularly their applications in communication networks and emerging technologies, including DL in SDN-based wireless sensor networks, delay-tolerant networks, AI in healthcare, network security, and AI ethics. Her previous research interests include on-chip networks and mathematical modeling for communication networks. She is a member of several IEEE societies.

SAMIA LOUCIF (Senior Member, IEEE) received the Bachelor of Engineering degree in computer science from Constantine University, Algeria, the master's degree in computer science from the University of Constantine (in collaboration with the University of Glasgow), and the Ph.D. degree in computer science from the University of Glasgow, U.K. She is currently an Associate Professor with the College of Technological Innovation, Zayed University, United Arab Emirates. Before joining



oratory, University of Blida 1. His research interests include computer networking, wireless networks, performance modeling, and computer architecture. He has served as the Editor-in-Chief for *Journal of Engineering Research*. He has also served on the Editorial Board of reputable journals, including IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, *International Journal of Parallel, Emergent and Distributed Systems*, *International Journal of Computers and Applications*, *International Journal of High Performance Computing and Networking*, and *Journal of Computer and System Sciences*.

MOHAMED OULD KHAOUA was a Professor with Sultan Qaboos University, Oman, from 2007 to 2015; a Reader with Glasgow University, U.K., from 2000 to 2006; a Lecturer with Strathclyde University, U.K., from 1997 to 2000; and a Postdoctoral Research Fellow with Teesside University, U.K., from 1994 to 1997. He is currently a Professor with the Department of Computer Science, University of Blida 1, Algeria. He is a member of the LRDSI Research Lab-

• • •